SPATIAL INFORMATICS · FINAL PRESENTATION

Large-Scale Terrain Data Analysis 2

Radar visibility, flight-route visibility and radar-hidden corridors over the Garhwal Himalaya: one dataset, one pipeline, one QGIS plugin

Track 1 Sravani **Track 2** Saloni **Track 3** Akshat

One terrain, three visibility problems

1

TRACK 1 · SRAVANI

Radar visibility

Which terrain, and which low-flying aircraft, can a ground radar see?

Naive line of sight, R2 ray casting, GDAL; range, height, many radars

2

TRACK 2 · SALONI

Flight-route visibility

What does an aircraft see along its route, and what does it miss?

Viewsheds along a route, aggregation, sampling, altitude, threads

3

TRACK 3 · AKSHAT

Radar-hidden corridor

Which route keeps the aircraft hidden from the radars, interactively?

Graph search on 16 M cells, precompute vs on the fly, QGIS plugin

Shared: DEMs, visibility model and radar sites (Track 1 → 3), route endpoints (Track 2 → 3); Track 3 scores every track's routes with one detection model.

Shared data and model: Garhwal Himalaya



Copernicus GLO-30 DSM, UTM 44N (EPSG:32644), 30 m cells. Four square DEMs centred on Rudraprayag: coverage grows, resolution stays fixed.

DEM	cells	extent	elevation
500 × 500	0.25 M	15 km	594–2,856 m
1000 × 1000	1 M	30 km	547–3,022 m
2000 × 2000	4 M	60 km	485–4,480 m
4000 × 4000	16 M	120 km	315–7,074 m

One visibility model for all tracks

Radar masts 30 m · aircraft 150 m above ground · 40 km range · Earth curvature with refraction k = 1/7 · six ridge-top radar sites

One pipeline, one dataset, one machine

Part	Steps (validation, scenario, every benchmark, analysis)	time	status
Track 1	validate, run_radar, benchmark, analysis	7.3 min	ok
Track 2	run_flight, benchmark, spacing_study, analysis	12.7 min	ok
Track 3	run_corridor, benchmark, analysis, interactive_bench, extensions_bench, efficiency_bench, panzoom_bench	48.0 min	ok
Cross-track + QGIS	cross_track, build_project, render_bench	1.0 min	ok*

`run.bat -m integration.pipeline` runs everything one step at a time in fresh processes, clears every benchmark CSV first and records commit 3074fca, the machine and each step's time.

How we measure: median of 5 runs, each in a fresh process (compute kept apart from I/O) · peak working set · complexity = slope of log(time) vs log(cells) · accuracy against synthetic terrains and exact references (agreement, kappa, Jaccard, cost gap).

AMD Ryzen 9 7900X, 12 cores / 24 threads, 31.1 GB · Python 3.12.13, GDAL 3.13.1, NumPy 2.4.6 · whole pipeline 69 min · *QGIS crashed on exit after writing all render results

TRACK 1 · SRAVANI

Radar visibility

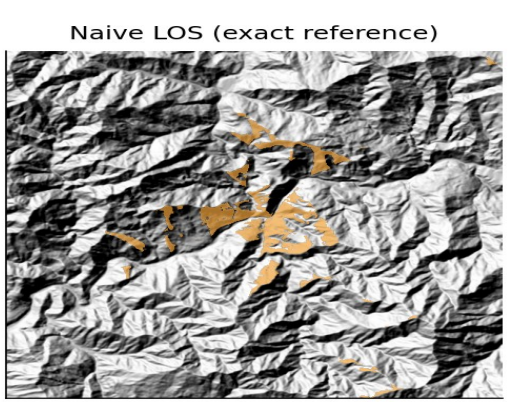
Which terrain, and which low-flying aircraft, can six ridge-top radars see? Three viewshed algorithms, validated and benchmarked.

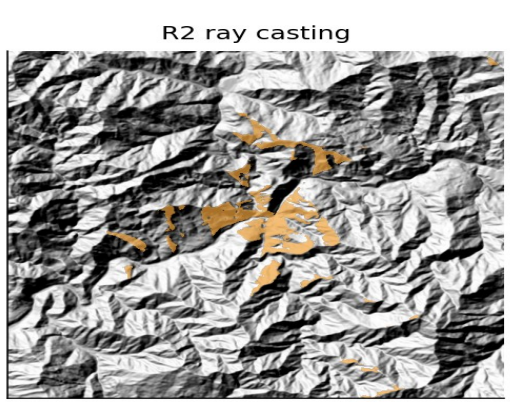
Three viewshed algorithms that agree with the exact answer

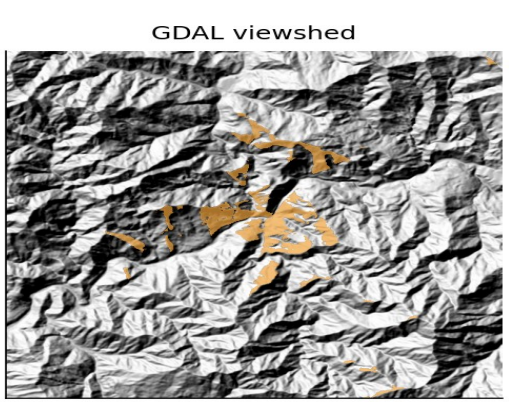
Algorithm	Idea	Cost
Naive line of sight	sample the profile to every cell; exact reference	O(N·√N)
R2 ray casting	one ray per boundary cell with a running horizon (own NumPy code)	O(N)
GDAL viewshed	C++ reference-plane algorithm	O(N)

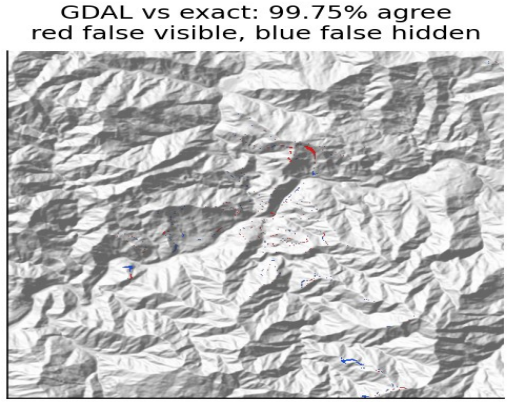
Real DEM vs exact	R2	GDAL
agreement	99.7%	99.8%
kappa	0.96	0.96

Synthetic flat, valley, wall and cone terrains: all three 100% correct. A valley test caught an R2 bug (84.7% → 100%).



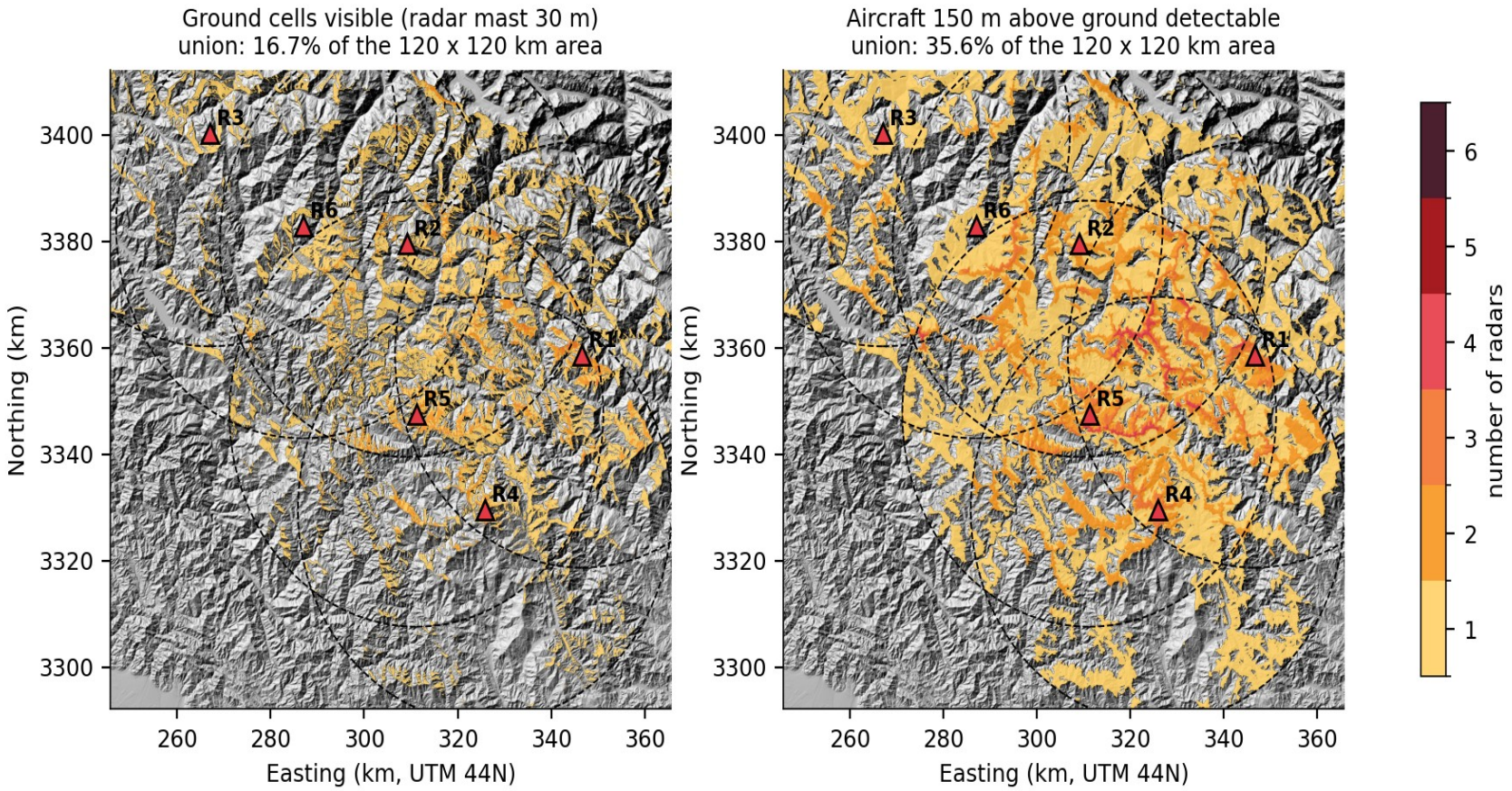






TRACK 1 · SCENARIO

Six radars see 16.7% of the ground



16.7%

of the ground seen by at least one radar

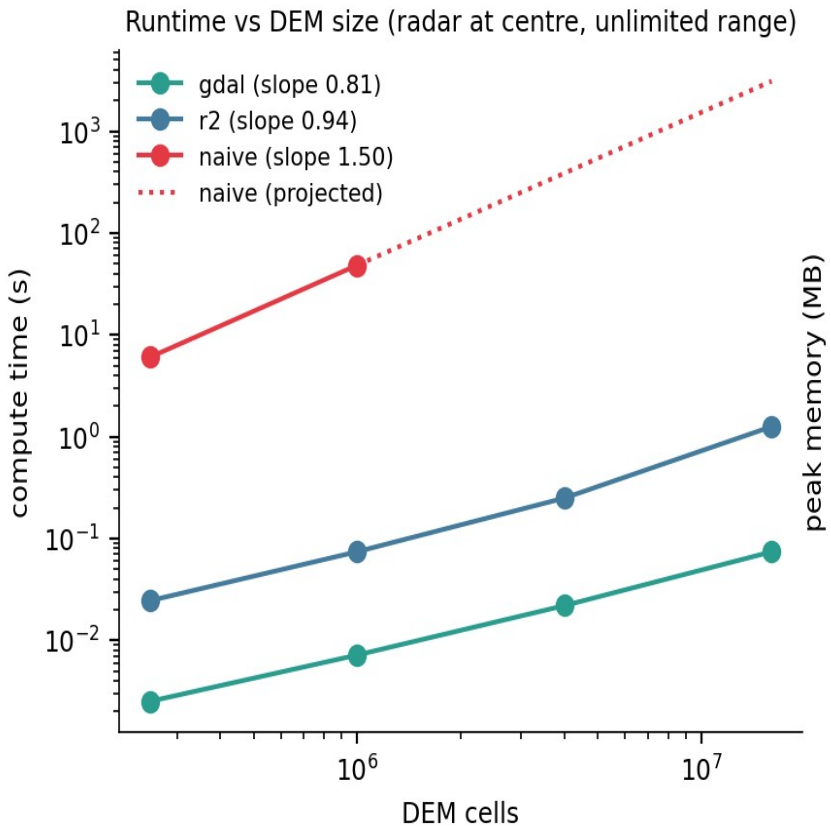
35.6%

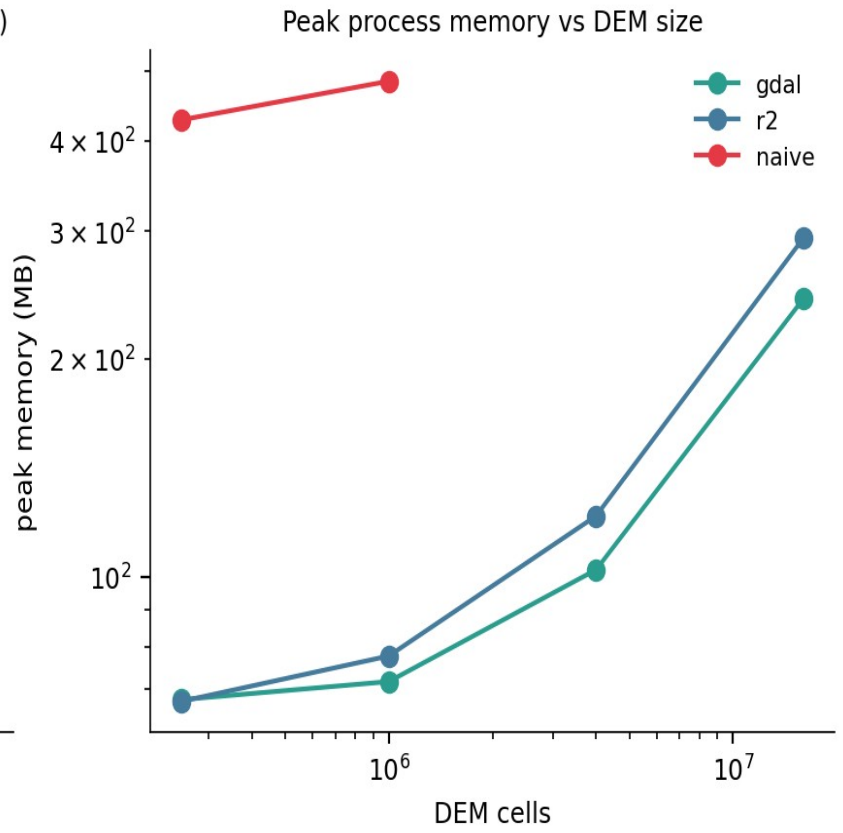
where an aircraft 150 m up is seen: Track 3's threat

Each radar sees 4%–14% of the ground within 40 km.

Sites: highest point within 3 km, 1,500–3,200 m, ≥ 22 km apart · 30 m masts · 40 km range

Exact costs N^1.5; the fast methods stay linear





51.6 min

exact method, 16 M cells (projected)

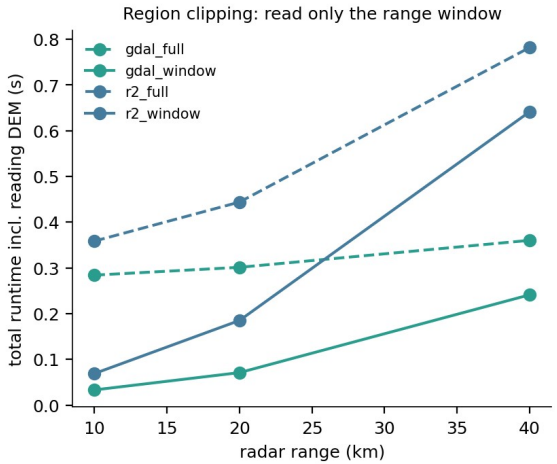
74 ms

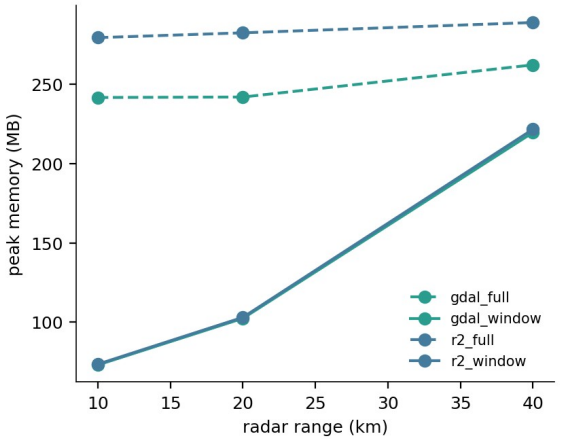
GDAL, same DEM

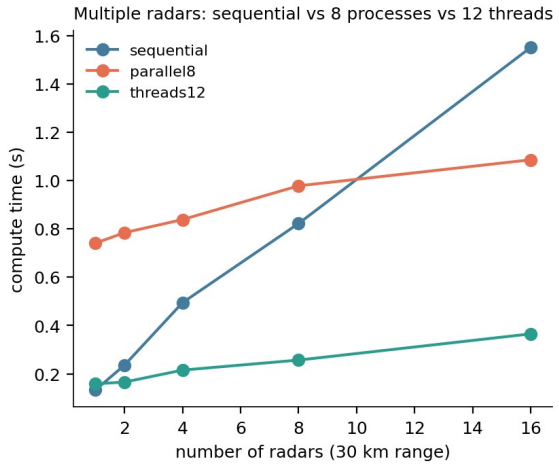
Fitted slopes: naive 1.50, R2 0.94, GDAL 0.81 (theory 1.5, 1, 1).

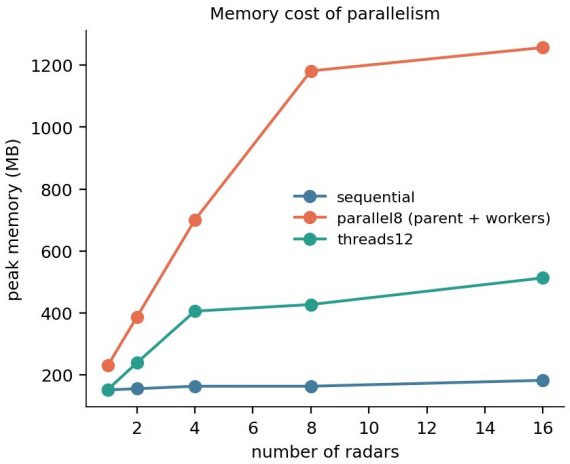
Radar at the DEM centre, unlimited range · GDAL peak memory 68 → 242 MB

Read less, use threads, coarsen with care









Study (16 M cells)	Result
E2 range 5 → 60 km	GDAL time grows as range^1.46 (area ~ range²)
E3 mast height	changes coverage, not cost
E4 many radars	12 threads: 4.2× faster at 16 radars; processes lose up to 8 radars (start-up)
E5 read only the range window	same output, a fraction of the time and memory
E6 coarser DEM 60 / 90 / 120 m	kappa 0.90 / 0.83 / 0.78 vs 30 m

TRACK 2 · SALONI

Flight-route visibility

A moving observer = many viewsheds, aggregated: what the aircraft sees, when it first sees it, and what it never sees.

Route, altitude, sample, aggregate

1 · Route

166 km up the Alaknanda valley, Devprayag → Joshimath (least-elevation path between towns)

2 · Altitude

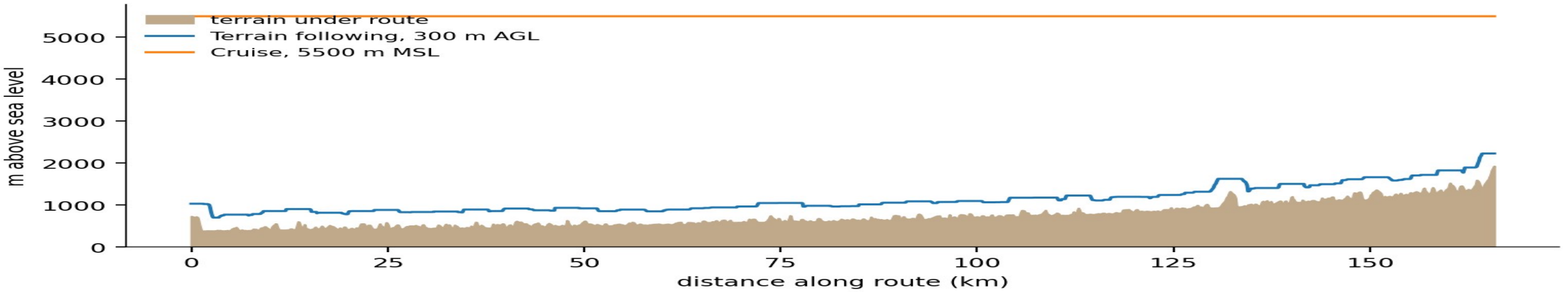
terrain following 300 m above smoothed ground, or a 5,500 m cruise

3 · Sample

a position every 1,000 m (167), a viewshed within 20 km at each

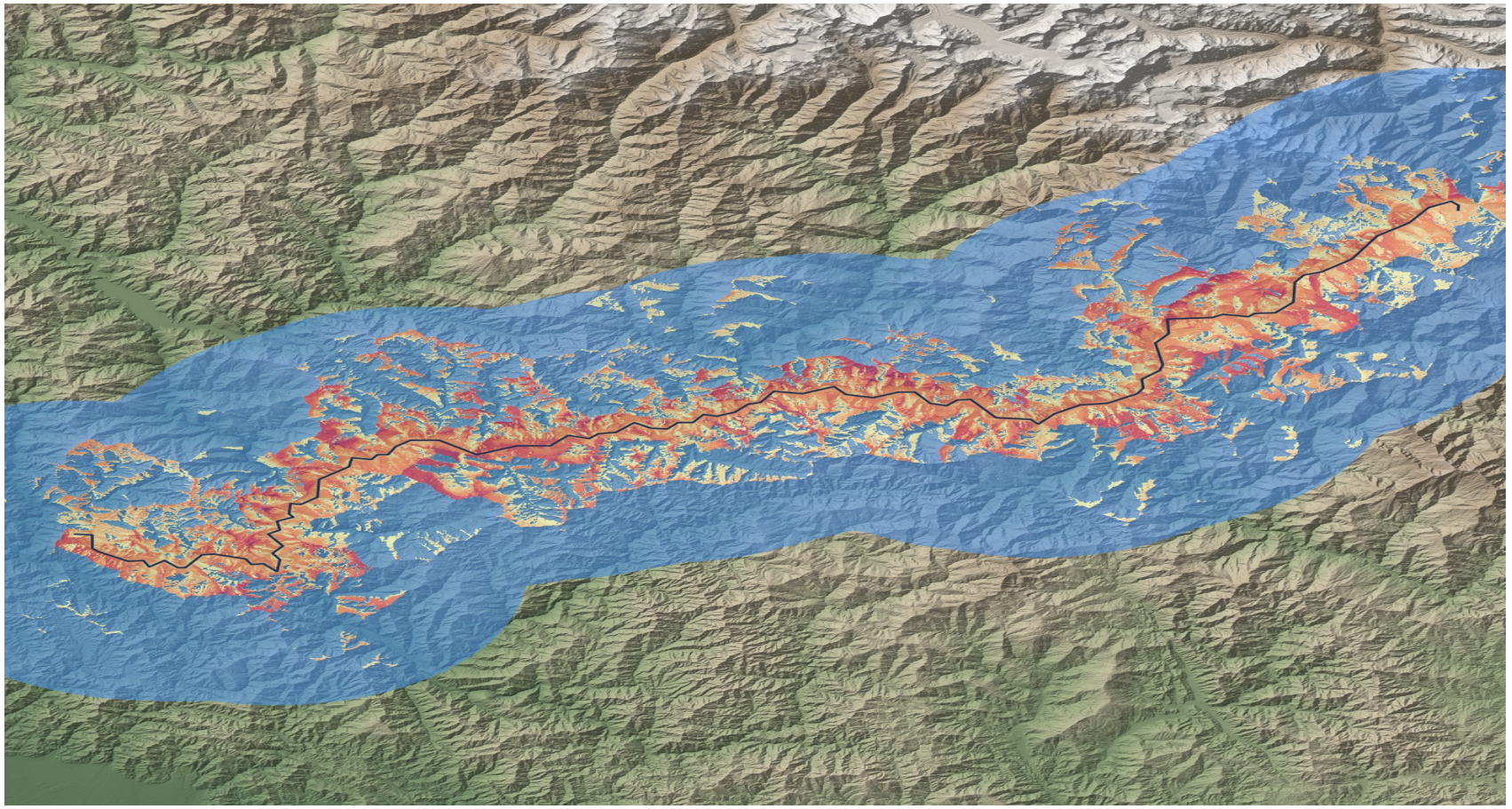
4 · Aggregate

times seen per cell, first position that sees it, new area per position, blind spots



TRACK 2 · SCENARIO

Valley flying sees 26% of the corridor



Corridor = 6,024 km² within 20 km of the route · blind spots = corridor cells never seen

1,537

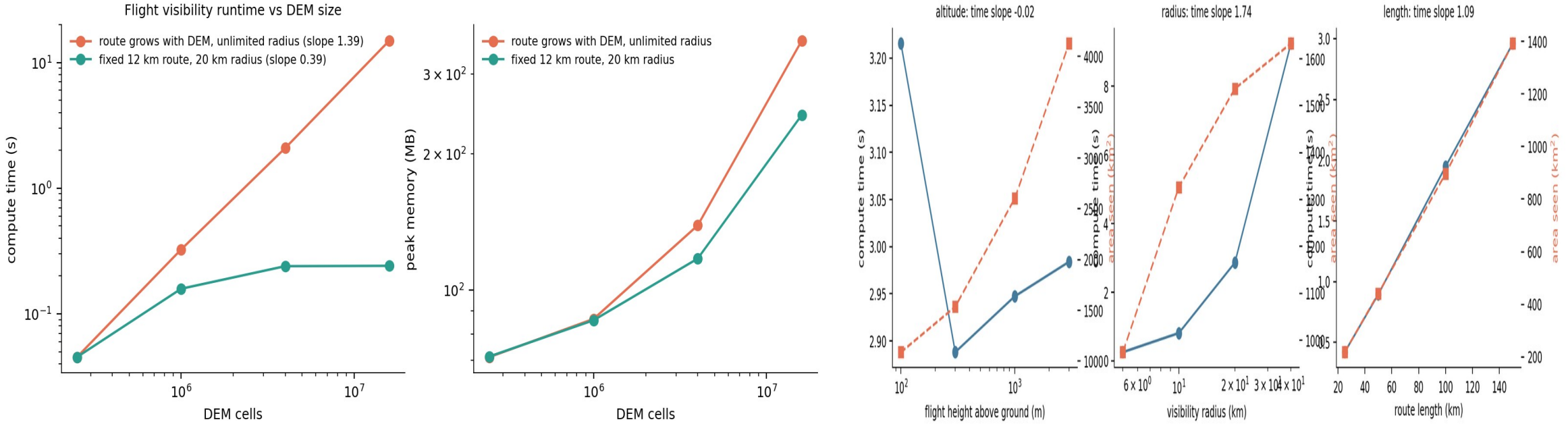
km² seen at 300 m AGL (26% of the corridor)

4,732

km² seen cruising at 5,500 m (79%)

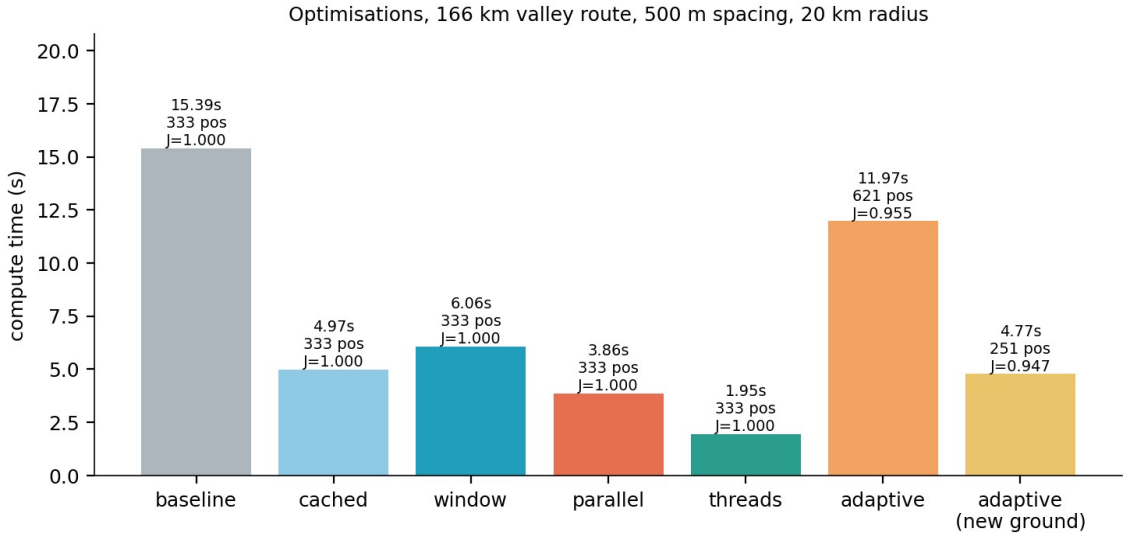
167 viewsheds in 2.9 s.

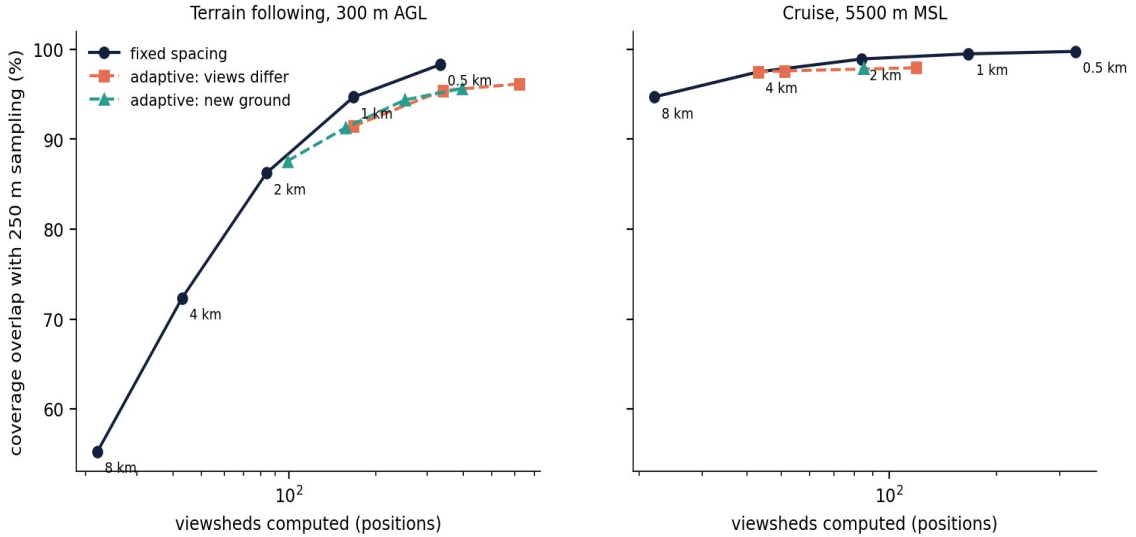
Cost = positions × cells per viewshed



Study	Result
F1 DEM + route grow	45 ms → 15.0 s, slope 1.39 (theory 1.5)
F1b fixed 12 km route	flat once the DEM is larger than the radius (slope 0.39)
F2 / F4 / F5	linear in positions (0.98), ~radius² (1.74), linear in length (1.09)
F3 altitude	no cost change (-0.02); coverage grows with height

7.0× faster with threads; spacing should follow altitude

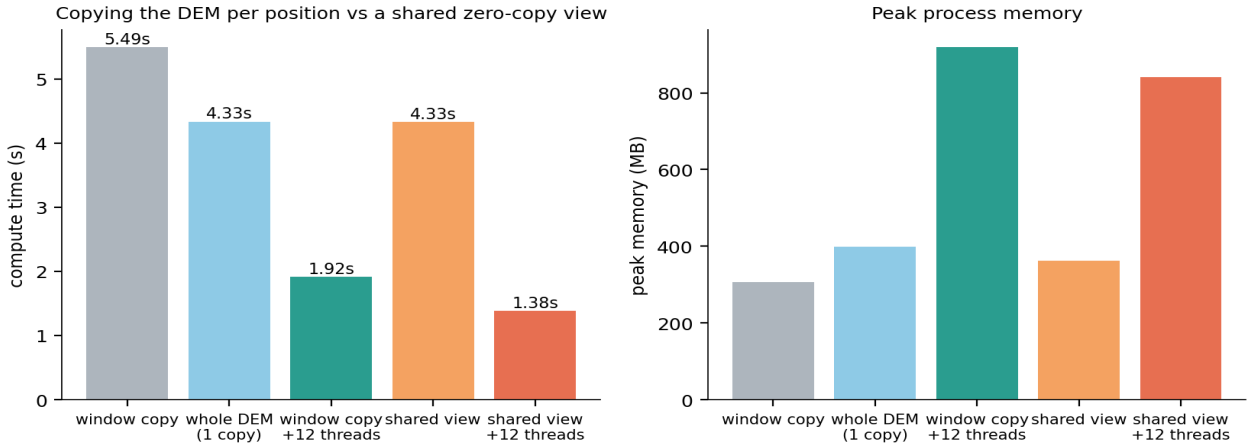




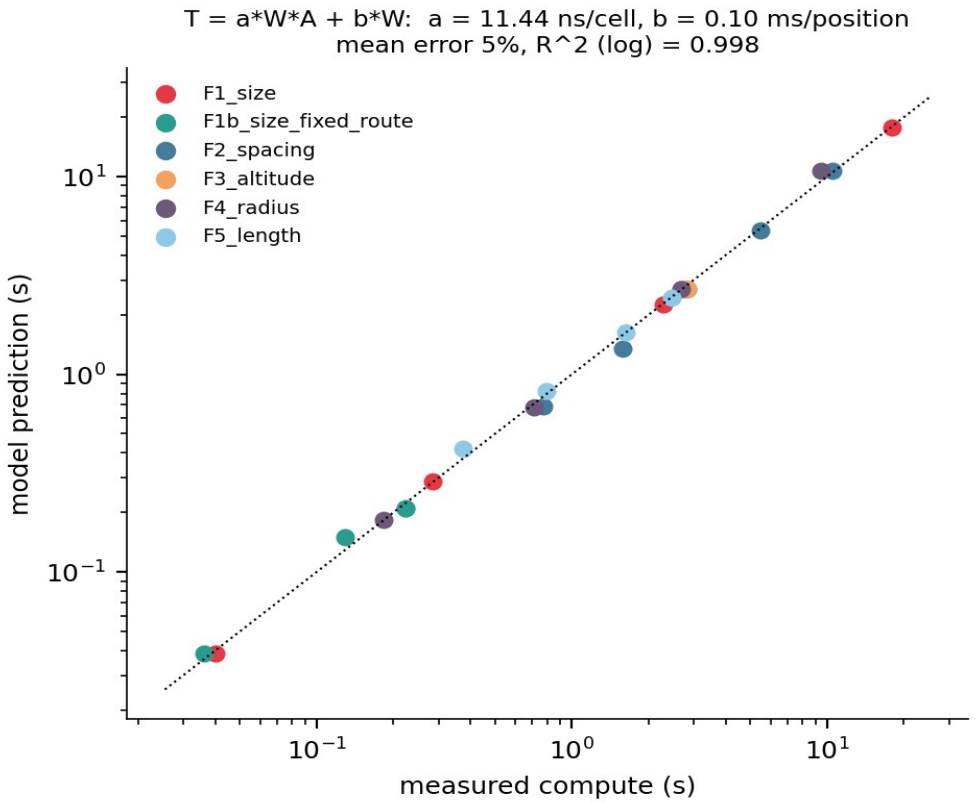
Speed: 333 viewsheds take 15.7 s re-reading the DEM, 5.3 s cached, 4.2 s with processes and **2.2 s with threads** (identical result).

Sampling: adaptive refinement never beat fixed spacing. The lever is altitude: at cruise 4 km spacing keeps ~97% of coverage; valley flying needs ~500 m.

Shared DEM: 1.38 s instead of 1.92 s, same answer

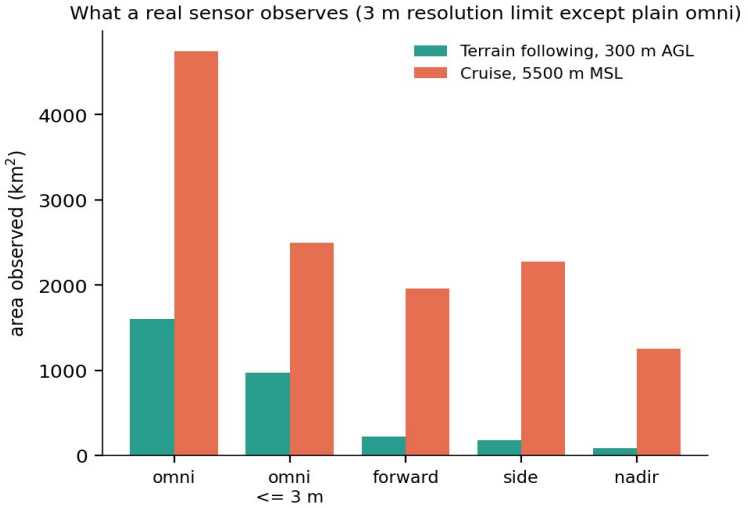


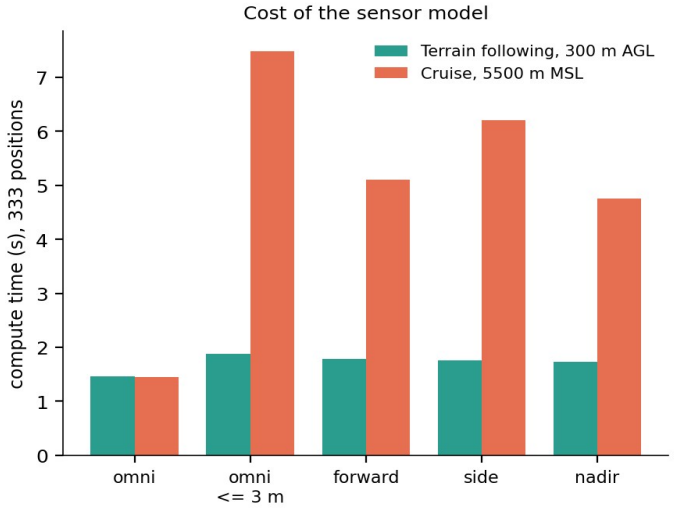
Idea	Result
F7 shared view + threads	1.38 s, 841 MB; identical result
F8 30 m near, coarse far	viewshed 2.3× faster, route ~10%; 86–97% overlap
F9 tiled accumulator	12 km route, 16 M cells: 92 → 10 MB of rasters
F10 T = a·W·min(πr², N) + b·W	a 11.4 ns/cell, b 0.10 ms/pos, error 4.9%

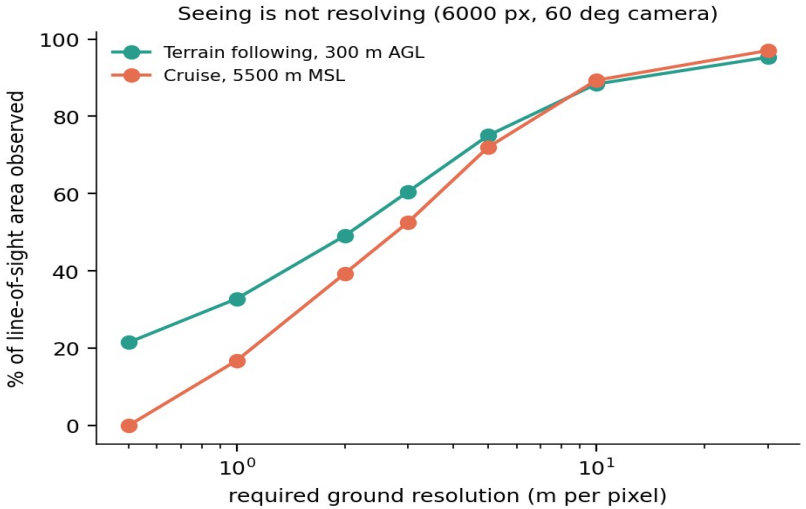


Linux laptop: 10 cores / 12 threads, 15.3 GB, GDAL 3.4.1 · results/linux/flight

A forward camera observes 220 of 1,600 km² seen





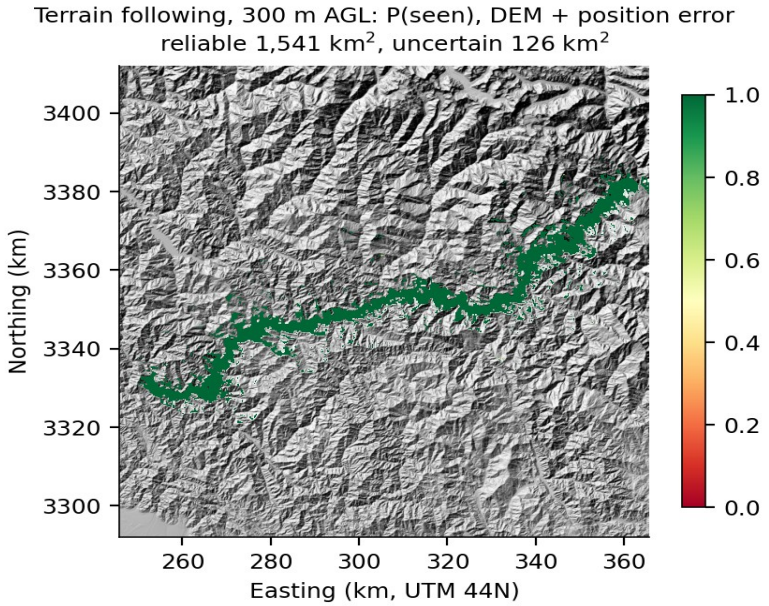


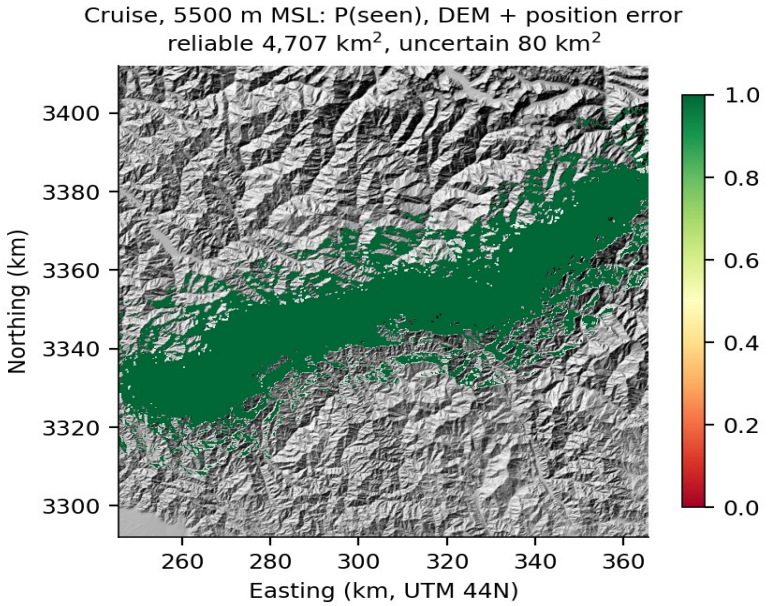
Sensor (3 m per pixel)	Valley 300 m	Cruise 5,500 m
line of sight only	1,600 km²	4,747 km²
forward camera 90°	220 km², 25 s	1,963 km², 50 s
side cameras 2 × 60°	175 km², 8 s	2,272 km², 67 s
nadir ≥ 45° down	87 km², 8 s	1,249 km², 125 s

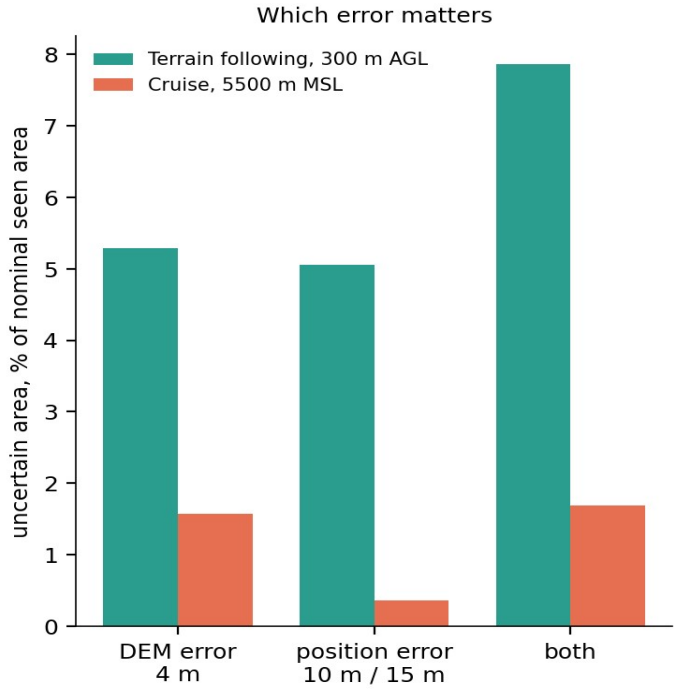
Field of view from the heading, depression limits, ground sample distance = slant × IFOV / cos(incidence), dwell = seconds observed at 60 m/s (median shown). Valley: forward is best (walls block the sides); cruise: side-looking.

Linux laptop: 10 cores / 12 threads, 15.3 GB, GDAL 3.4.1 · results/linux/flight

"Seen" is uncertain on 7.9% (valley), 1.7% (cruise)

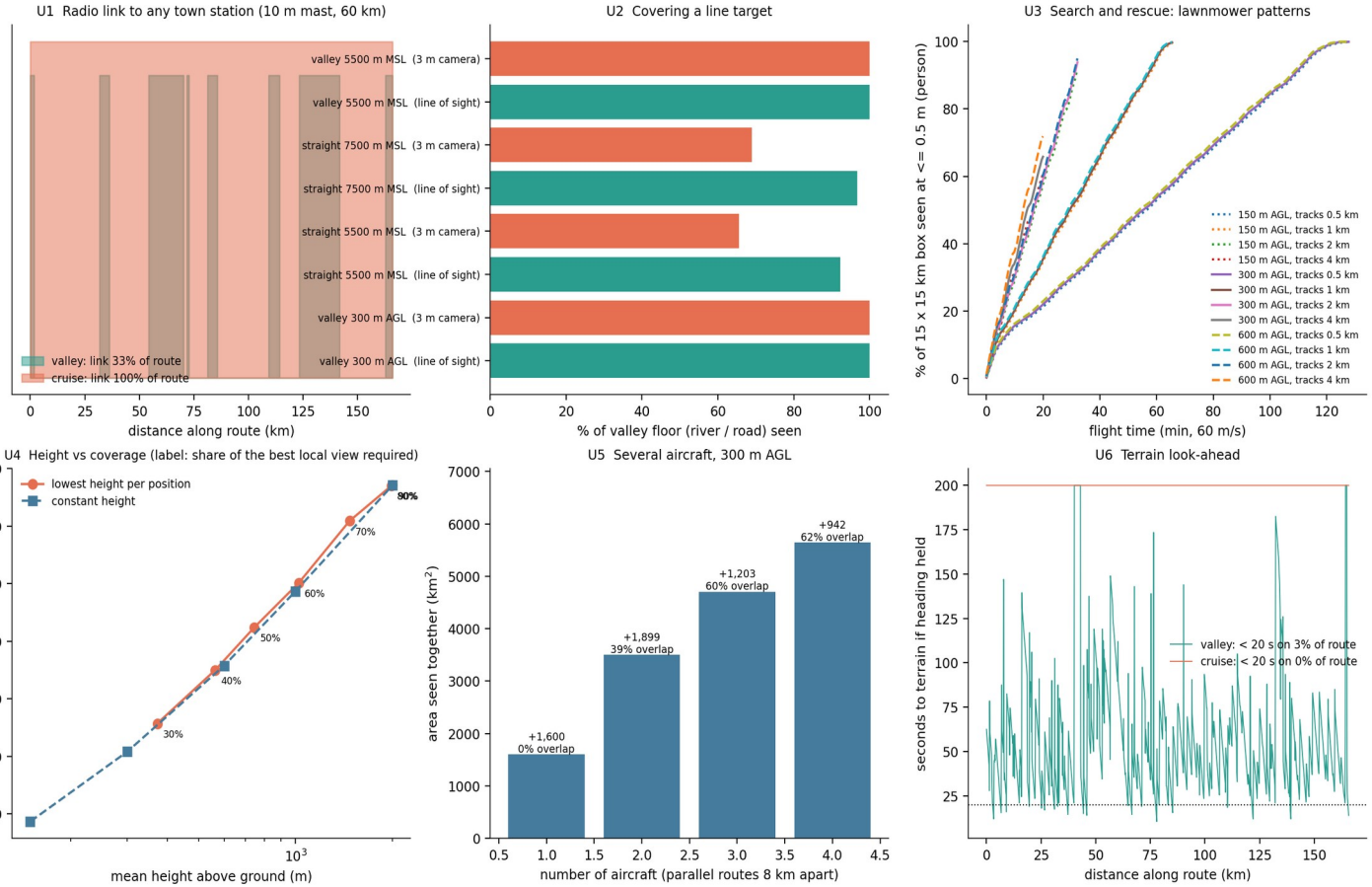






20 runs per source: DEM error 4 m (correlated over 300 m), aircraft position error 10 m horizontal / 15 m vertical. The total area barely moves (valley 1,604 ± 3 km²) but which cells are seen does, at the edges of visibility. Valley: DEM and position errors matter equally (85 / 81 km²); cruise: almost only the DEM.

Six applications of the same engine



Radio link: valley flight reaches a town station on 33% of the route (longest gap 30 km); cruise 100%.

River / road: following the valley sees all of it; a straight 5,500 m line only 65% with a 3 m camera.

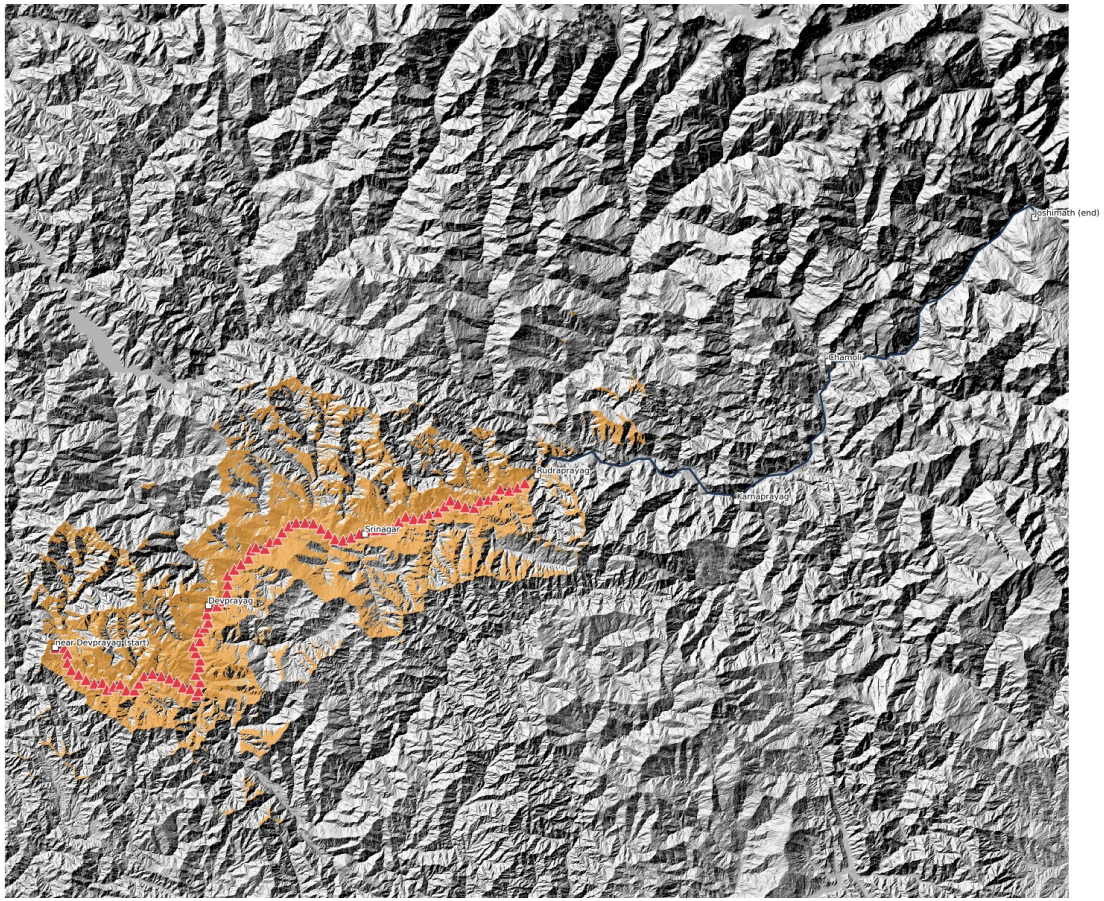
Search & rescue: 0.5 m camera, 15 × 15 km: 1 km tracks cover 99.98% in 66 min, 4 km tracks 66% in 20 min.

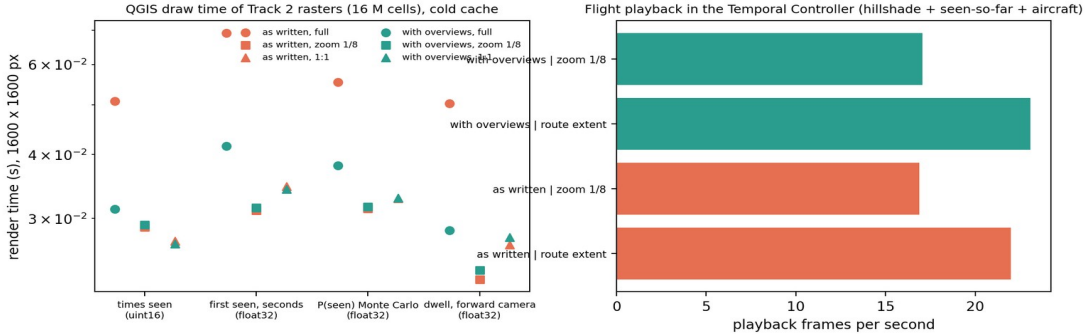
Height: 80% of the local view needs ~2 km above ground; per-position height gains only ~2%.

Several aircraft: 1,600 → 3,499 → 4,702 → 5,644 km²; the 4th overlaps 62%.

Look-ahead: < 20 s to terrain on 3.4% of the valley route if heading is held.

QGIS playback at 17–23 frames per second





How: the first-seen raster stores the flight time at which each cell is first seen (raster temporal mode 'represents temporal values'); the aircraft layer has a time per position; a cumulative range makes seen ground accumulate.
Draw time: Track 2 rasters (16 M cells) draw in 23–69 ms; overviews make full-extent ~1.6× faster and change nothing zoomed in.
Files: qgis/track2_extensions.qgz, qgis/track2_layers.py

TRACK 3 · AKSHAT

Radar-hidden corridor, interactively

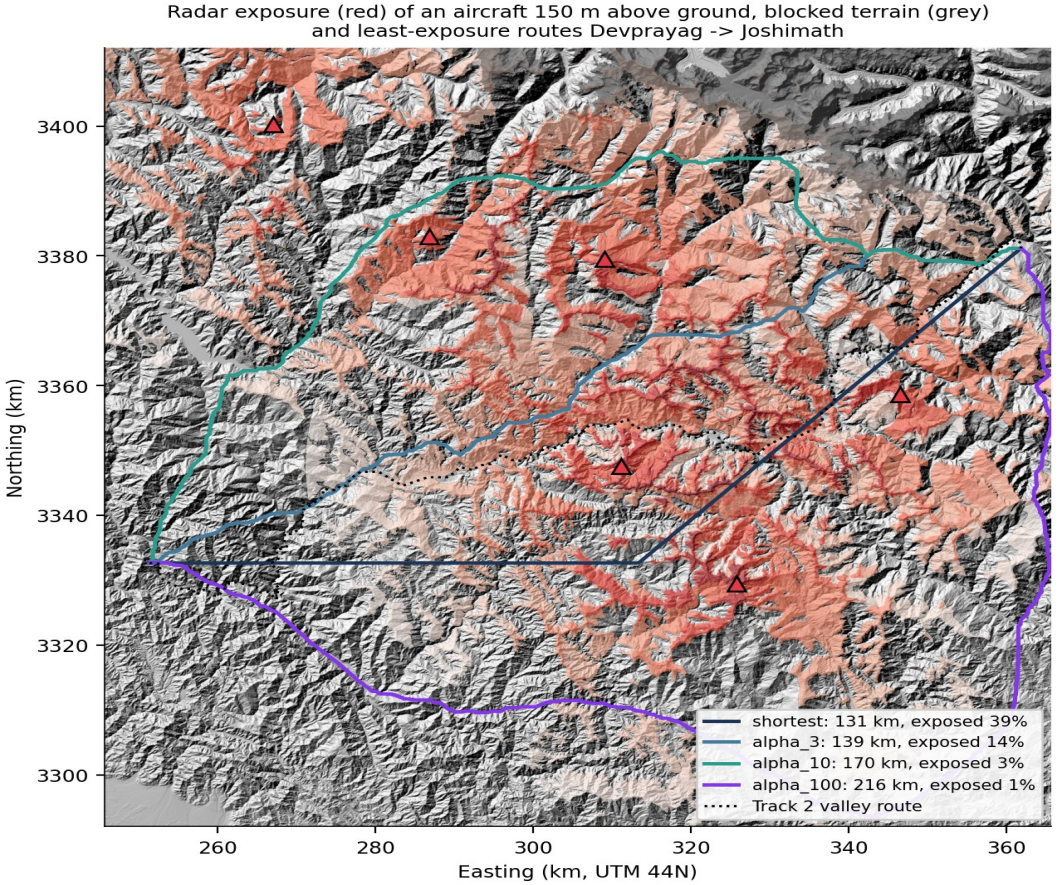
Exact routing on 16 M cells, what to precompute and what to compute on the fly, a certified risk budget, and the whole thing as a QGIS plugin (live demo at the end).

TRACK 3 · PROBLEM

Stay hidden from the radars without a big detour

$$c(u,v) = L \cdot (1 + \alpha \cdot \text{threat}) + \beta \cdot \text{climb}$$

Grid graph: 16 M cells, 8 neighbours, 128 M edges; aircraft 150 m above ground; above a 5,000 m ceiling is blocked.
Threat: from Track 1's radar viewsheds: exposure (visible, closer = worse) or detection hazard −ln(1 − p) per km.
Detection: P(detect) = 1 − exp(−Σ length · hazard): additive, so every shortest-path method applies.



30% longer, 93% less exposure

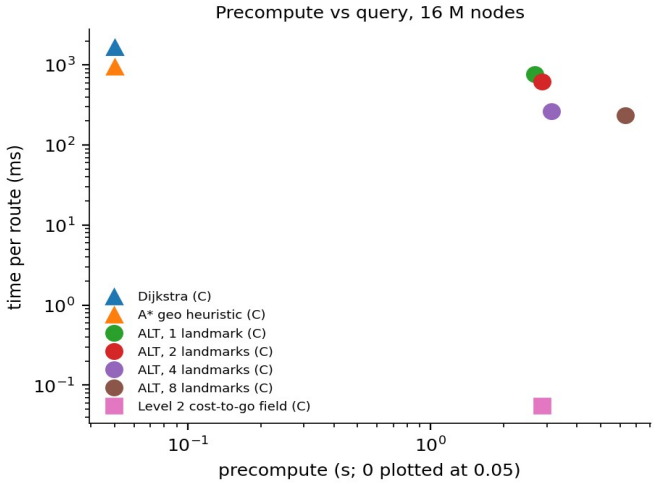
Devprayag → Joshimath	length	exposed	exposure
Shortest (α = 0)	131 km	39%	31.5
Track 2 valley route	178 km	26%	26.5
α = 3	139 km	14%	8.1
α = 10	170 km	3%	2.2
α = 100	216 km	1%	0.5

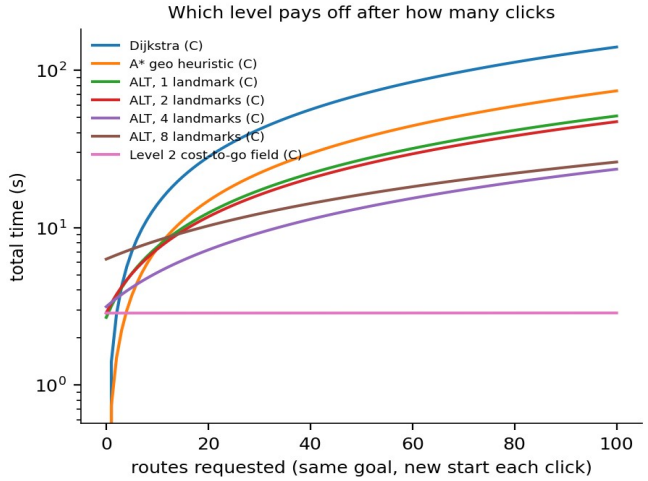
16 M nodes, one route	time	peak memory	gap
Dijkstra (Python)	37.7 s	682 MB	0
A* (Python)	28.2 s	1,031 MB	0
Dijkstra (SciPy)	6.4 s	6,025 MB	0
Multi-resolution ×4	2.1 s	1,217 MB	1.4%
Our C Dijkstra (implicit grid)	2.5 s	635 MB	0

All exact solvers return the same optimum. Fitted slopes 0.97–1.04 (theory N log N). The C search on the implicit grid needs no edge list: SciPy's 128 M-edge matrix costs 5.9 GB.

Exposure = km of route weighted by threat · ALT (landmark A*) returns the same cost and searches a fraction of the nodes

Precompute the field for the goal; every start is a lookup





Level (16 M cells)	Precompute	New start	New goal
0 · C Dijkstra	–	1,404 ms	1,678 ms
1 · 4 landmarks (ALT)	3.1 s · 488 MB	203 ms	264 ms
2 · cost-to-go field	0.28 s · 183 MB	0.06 ms	new field

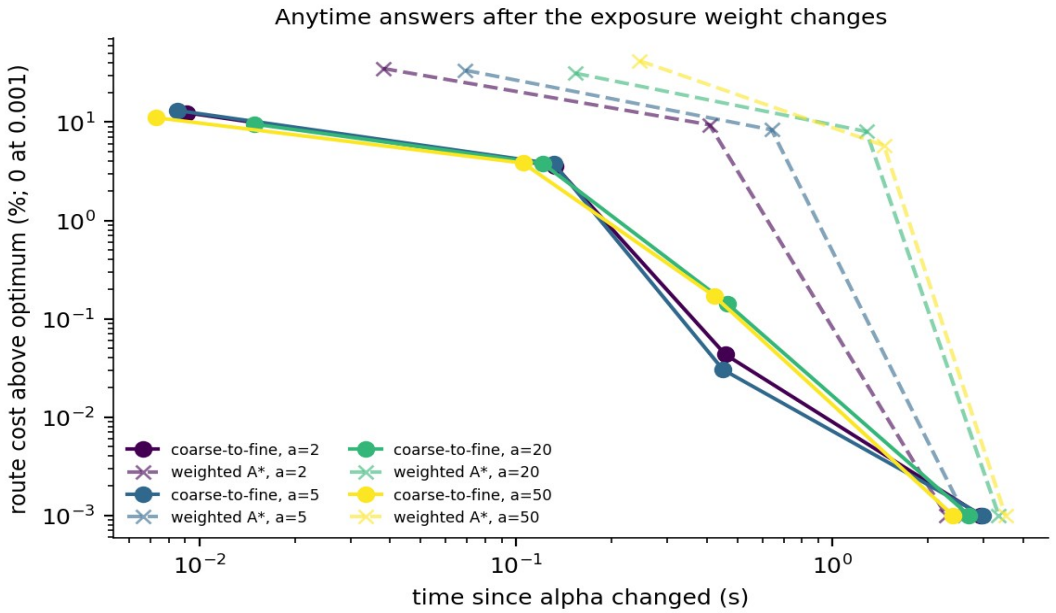
Cost-to-go field

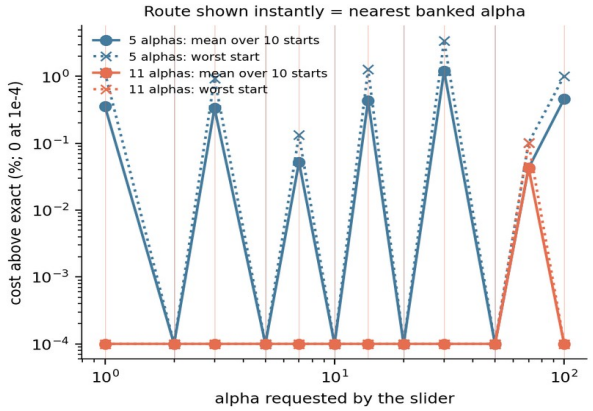
For every cell: cost of the best route to the goal and the next step. One backward search builds it; a route from any start is then a lookup: exact, no search.

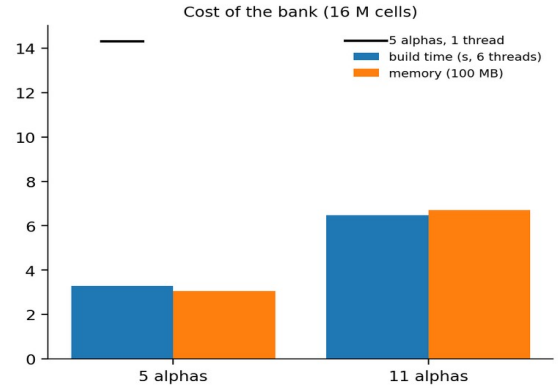
12 cores: 2.85 → 0.28 s

Delta-stepping with a lock-free atomic minimum, exact. 10.2× on 16 threads; a spin barrier fixed MinGW's sleeping one.

Answer at once, exact soon after



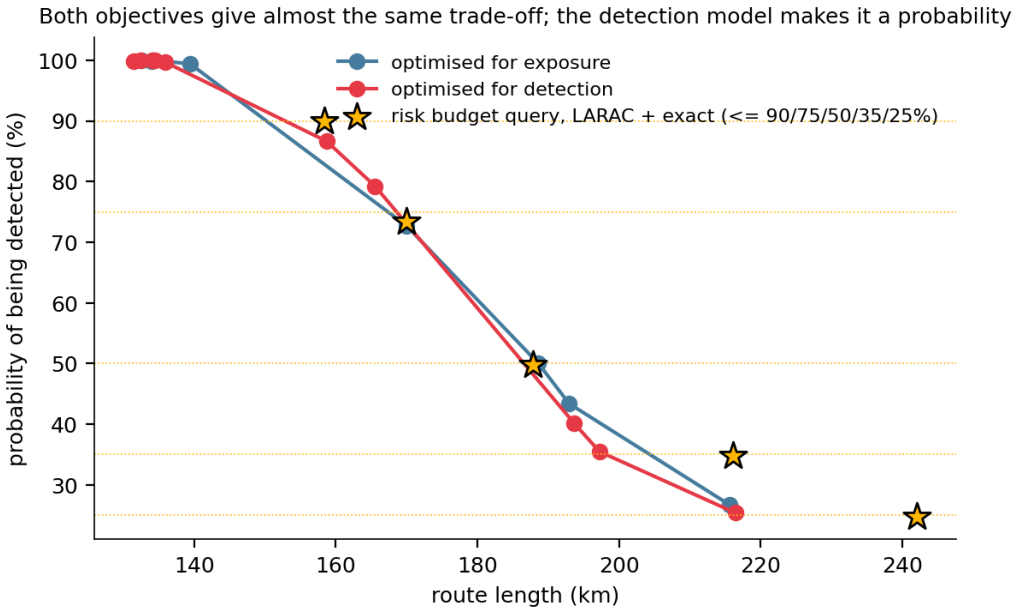


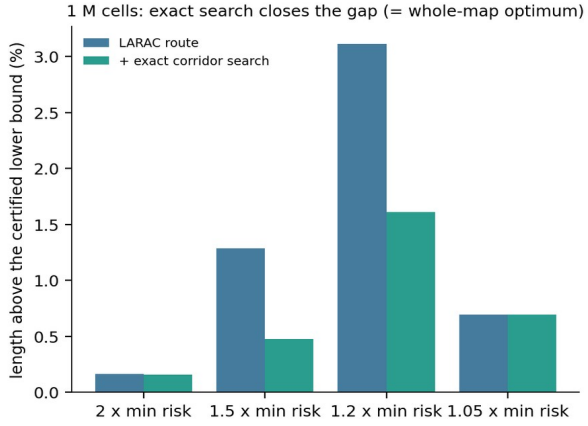


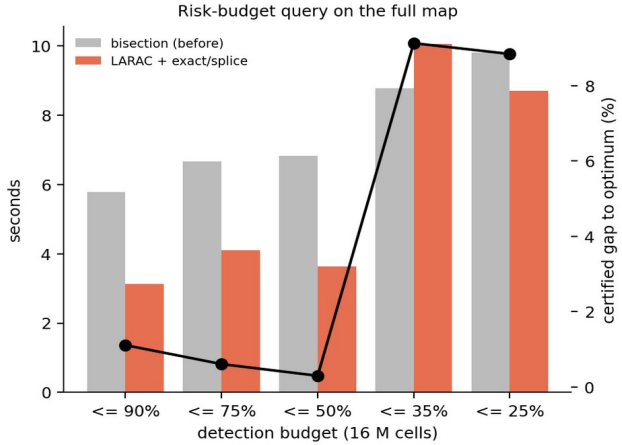
The pyramid stores threat and height, not costs, so it survives an α change; the field and landmarks do not. Weighted A* lost to coarse-to-fine at every quality level.

Interaction (16 M cells)	Strategy	Latency
Click a new start	Level 2 lookup	0.09 ms
Change α	α bank lookup → exact field in background	~1 ms → ~0.4 s
Drag / add a radar	that radar's viewshed + parallel field rebuild	~0.6–0.9 s
New goal	coarse-to-fine route now, field in background	~0.3–0.6 s
Pan / zoom	draw the known route (never re-search)	< 0.1 ms

Shortest route under a detection budget, with a certificate



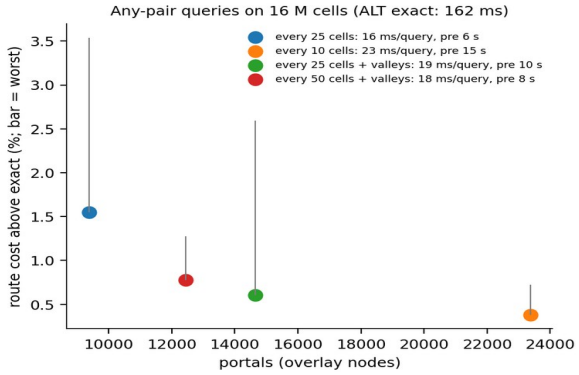


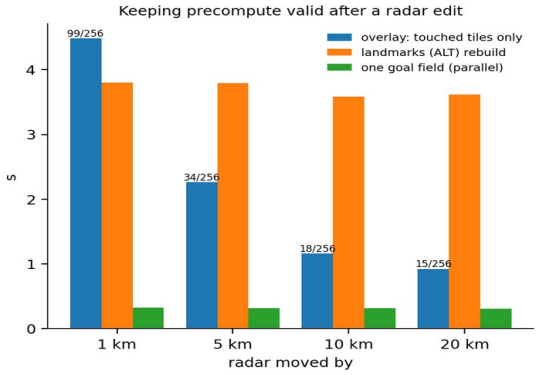


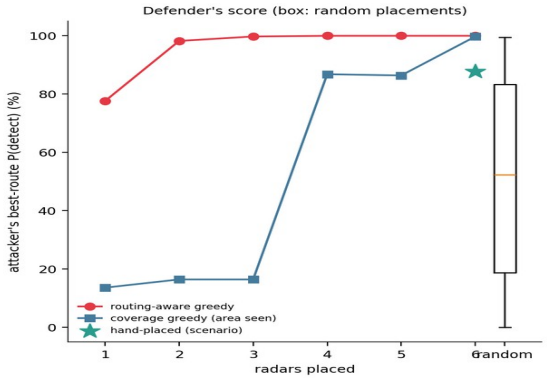
Resource-constrained shortest path (NP-hard): LARAC Lagrangian relaxation on parallel fields (2.7–4.7 s), then an exact label-setting search in a corridor with Lagrangian pruning and dominance. **The lower bound certifies how far the answer can be from the best possible length.**

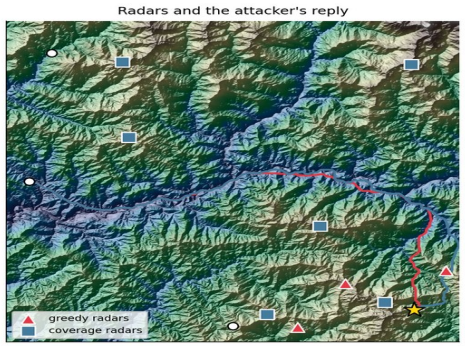
Exact on 1 M cells (corridor answer = whole-map optimum) · 16 M cells, P(detect) ≤ 50%: 188 km at 49.8% (plugin scenario: 145 km, within 0.4% of optimal) · Monte Carlo = formula

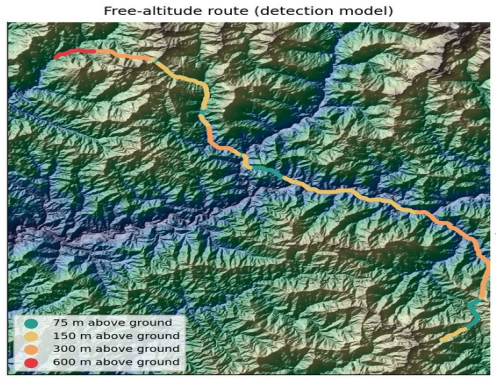
Tiles, radar placement, altitude, robustness

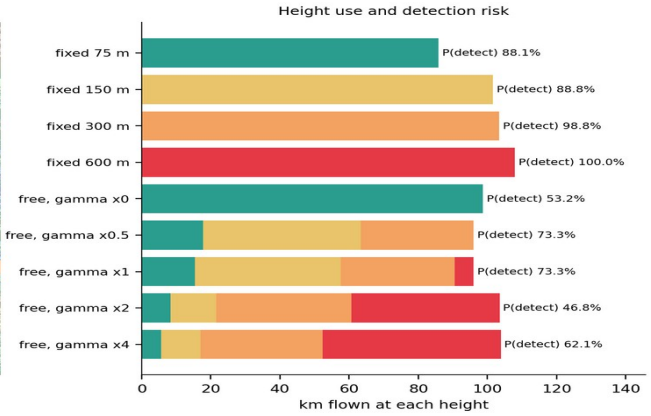


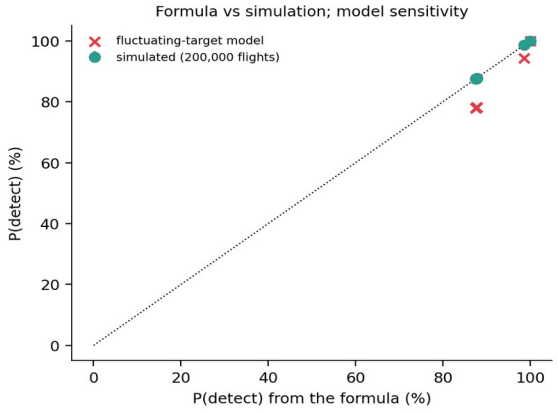


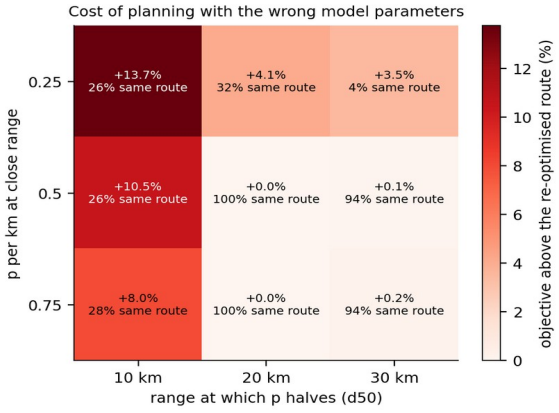






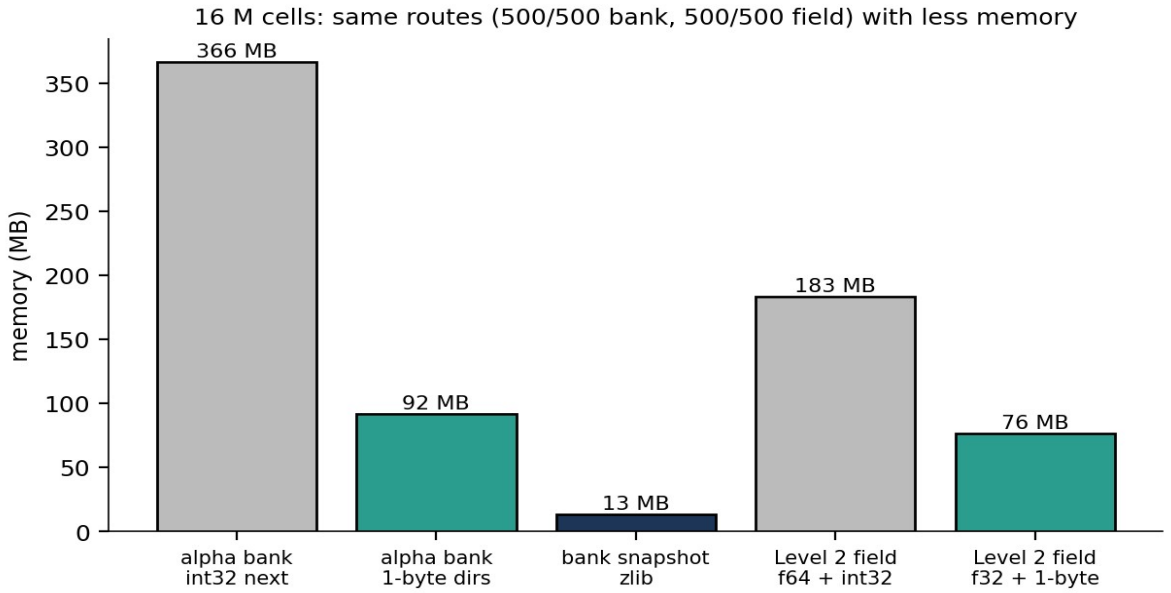




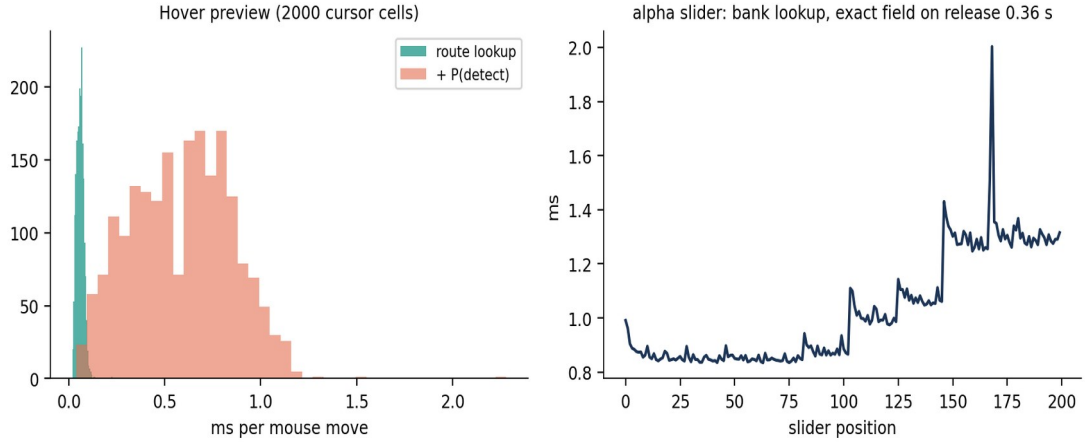


Tiles (CRP idea): any pair in ~19 ms, 0.6% above exact · placement for routes beats placement for coverage (2 radars: 98% vs 16% detection) · free altitude: P(detect) 89% → 53%

Same routes, a quarter of the memory; live previews

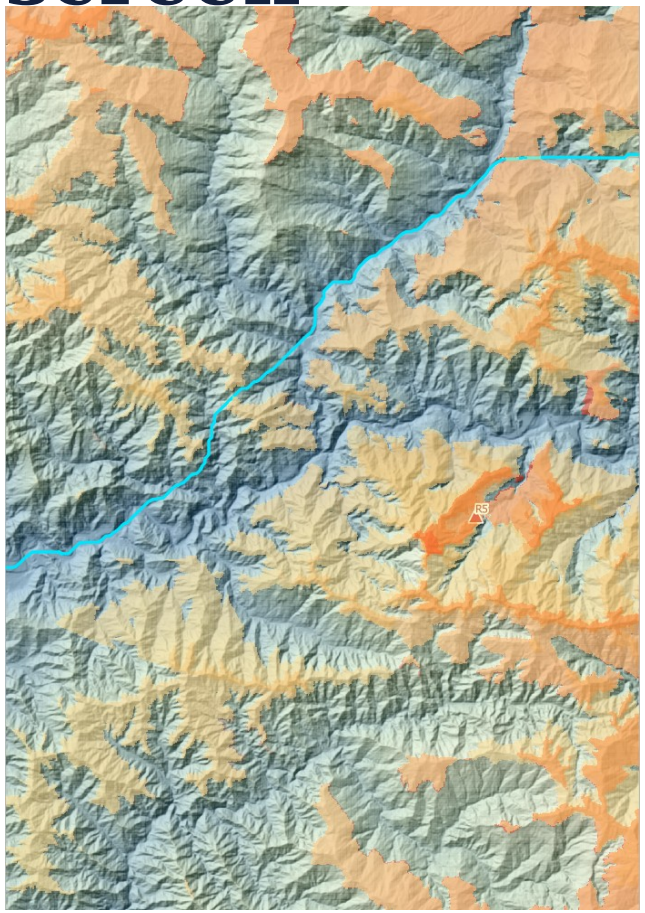


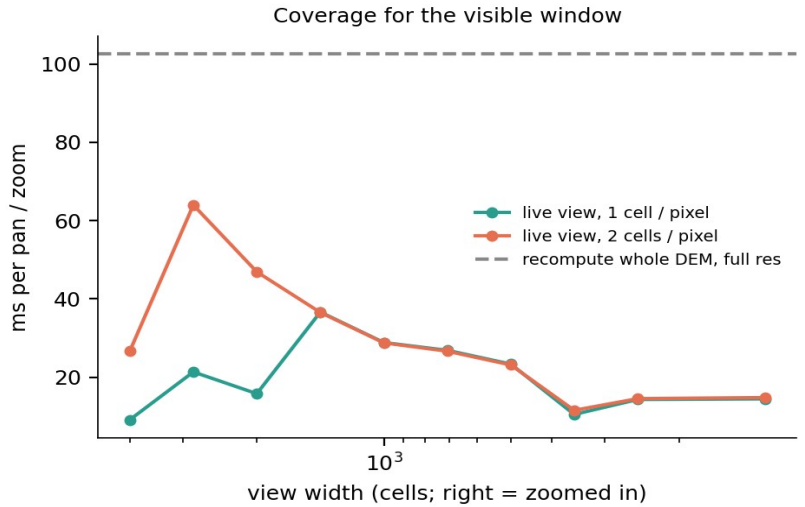
Memory (16 M cells)	before	after	routes
α bank, 6 fields: next → 1-byte direction	366 MB	92 MB	500/500
Field: f64 cost + int32 → f32 + 1 byte	183 MB	76 MB	500/500
Bank snapshot on disk (zlib)	92 MB	13.4 MB	lossless

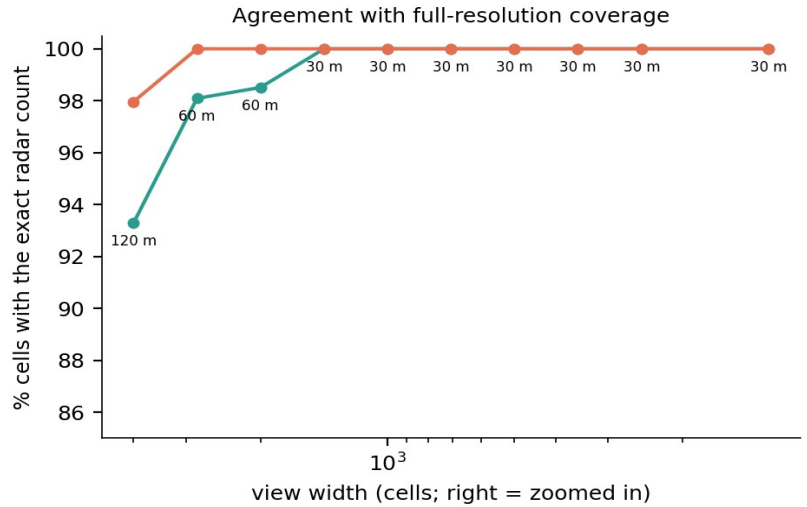


On the fly (plugin)	latency
Hover: route from the cursor	0.06 ms (p99 0.11)
… + P(detect) of that route	0.6 ms
α slider while dragging (bank)	0.94 ms
α slider released (exact field)	358 ms

Every pan and zoom recomputes only what is on screen



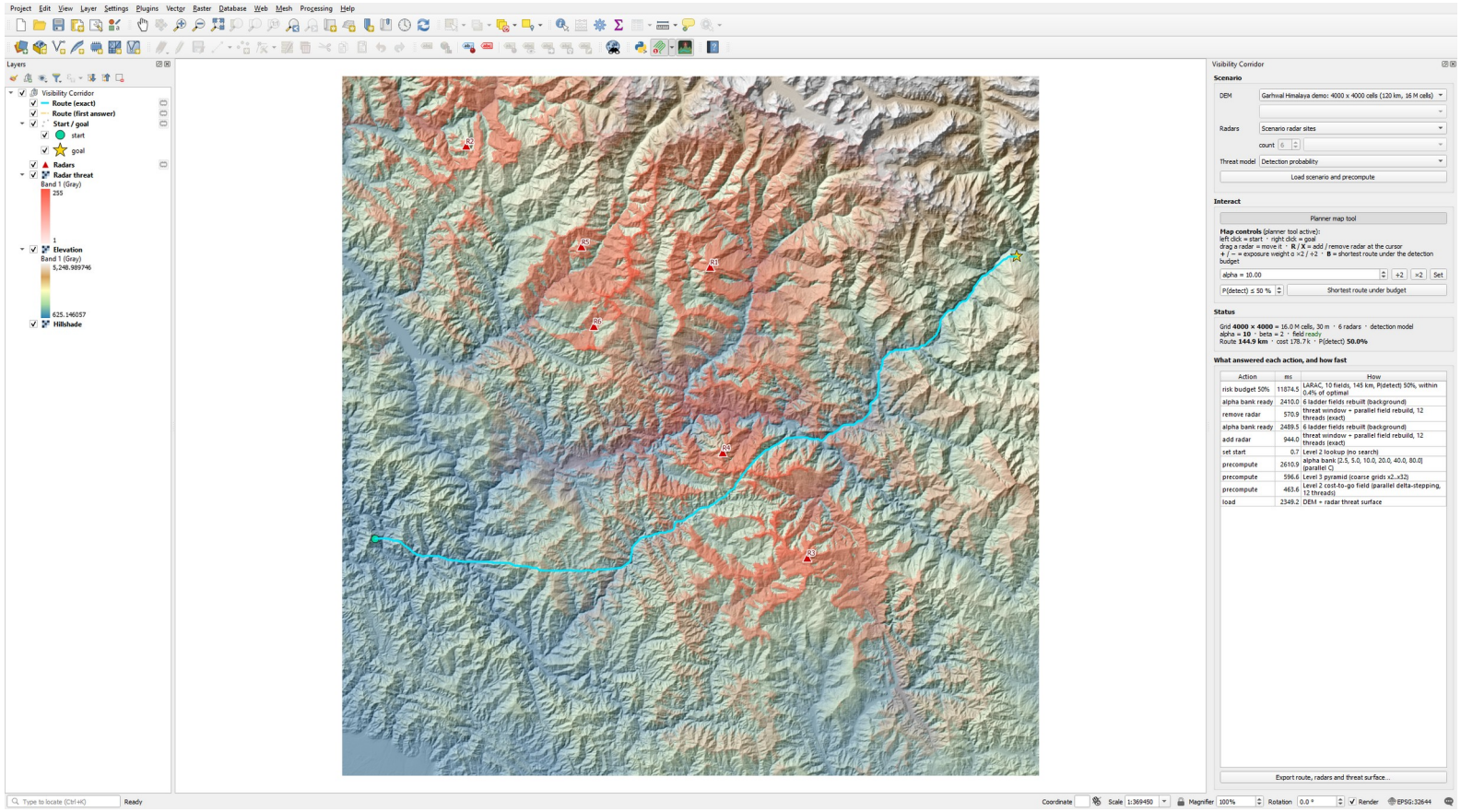




Per pan / zoom (canvas 1600 × 1000 px, 16 M cells)	time	exact cells
Recompute the whole DEM at 30 m	103 ms	100%
Live, zoomed out: 120 m level	9 ms	93.3%
Live, zoomed in (1414 cells wide or less): 30 m	23 ms	100%
All 73 views (zoom × pan)	median 20, max 40 ms	–
48 radars: whole DEM vs live, full extent	541 vs 51 ms	–

Level of detail: DEM pyramid 30 m → 1.9 km (built at load in 0.20 s, +20 MB) · one GDAL viewshed per radar, only on the box between radar and view, on 12 threads · stale views dropped while panning

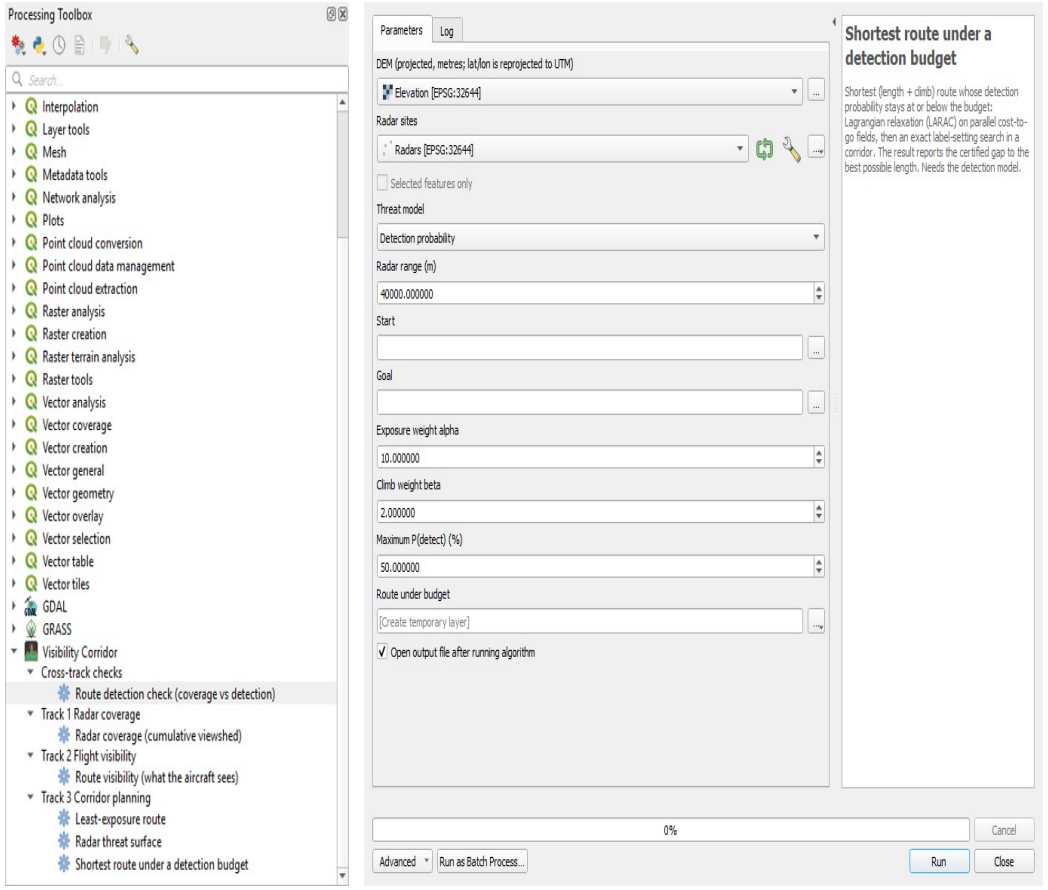
A QGIS plugin: the planner inside QGIS



Install: Plugins → Install from ZIP (Windows; not open source).
Map: DEM, radar threat, radars, start/goal and routes are real QGIS layers.
Controls: left / right click start / goal, drag a radar, R / X add / remove, + / − α, B risk budget, live hover preview, α slider; wheel / middle-drag pan and zoom with live radar coverage of the view.
Panel: which level answered each action and how long it took.
Any DEM: bundled 1000 / 2000 / 4000 scenarios, or any DEM layer (lat/lon reprojected to UTM), radars from a point layer.

Export: route, radars and start/goal to GeoPackage, threat surface to GeoTIFF · the C solver ships as a DLL, without its source

Our best algorithm per track as Processing tools



Tool	Track	Algorithm
Radar coverage	1	GDAL viewshed per radar, count
Route visibility	2	threaded window viewsheds (F6 best)
Radar threat surface	3	detection hazard or exposure
Least-exposure route	3	parallel C cost-to-go field, exact
Shortest route under a detection budget	3	LARAC + exact label search, certified
Route detection check	all	P(detect) + part outside coverage (= 0)

Usable in the Model Designer, batch mode and from Python, like any QGIS algorithm.

LIVE DEMO

Visibility Corridor in QGIS

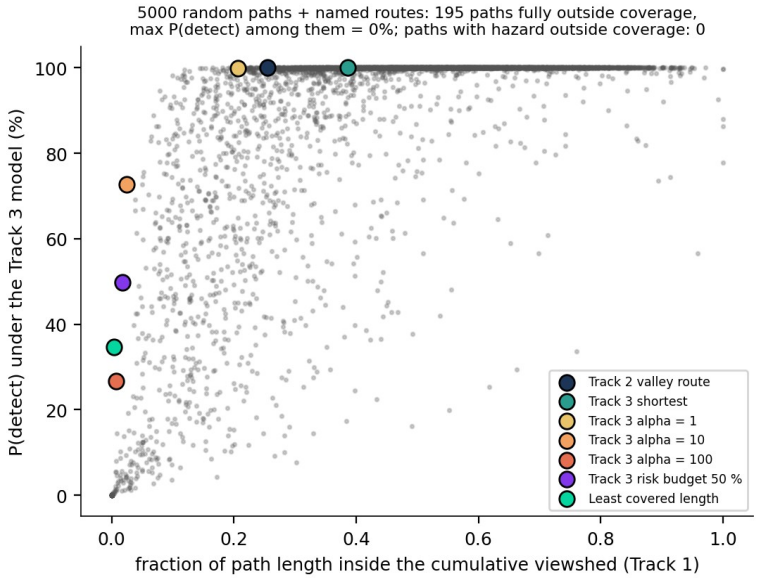
1 · Load the 120 km Garhwal scenario (~6 s) 2 · click starts, move the mouse: live preview 3 · right-click a goal 4 · drag a radar, R / X 5 · α slider, + / − 6 · B: route under 50% detection 7 · pan and zoom: live coverage 8 · Processing tool on another DEM

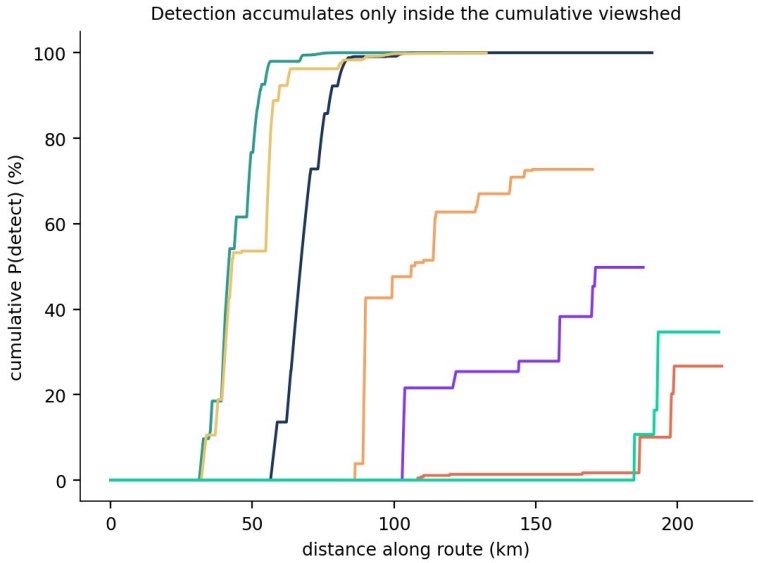
INTEGRATION

All three tracks on one dataset

Does every track agree with the others? Detection is checked against coverage, every route is scored by all tracks, and all benchmarks run on one machine.

No detection outside the cumulative viewshed

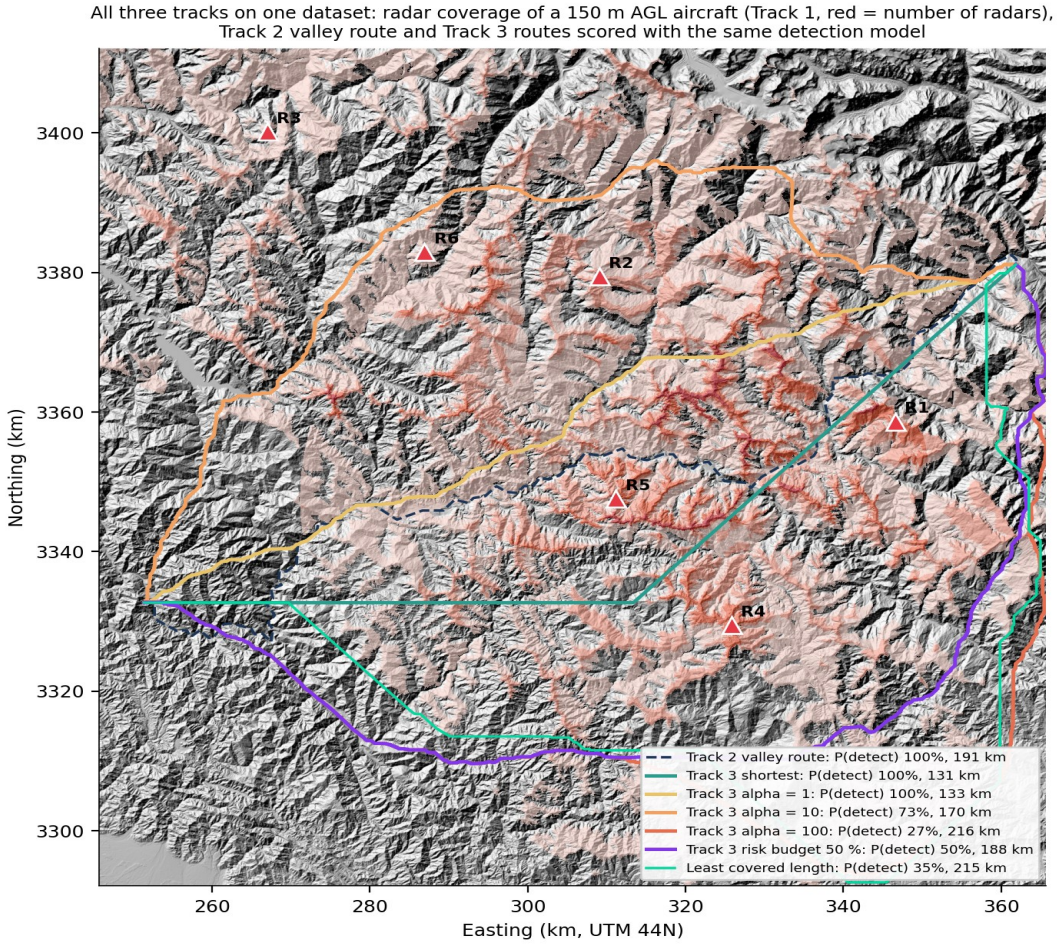




Check (16 M cells)	Result
Cells: Track 1 coverage = Track 3 hazard > 0	100.000% agree
Hazard outside coverage	0 cells
Random paths fully outside (of 5,000)	195, max P(detect) 0.00%
Paths with hazard outside coverage	0
Fully hidden Devprayag → Joshimath route	none

A path the radars cannot see cannot be detected: the detection hazard is zero wherever Track 1's coverage of a 150 m aircraft is zero, checked cell by cell, on every named route and on thousands of random paths. The same check is a Processing tool.

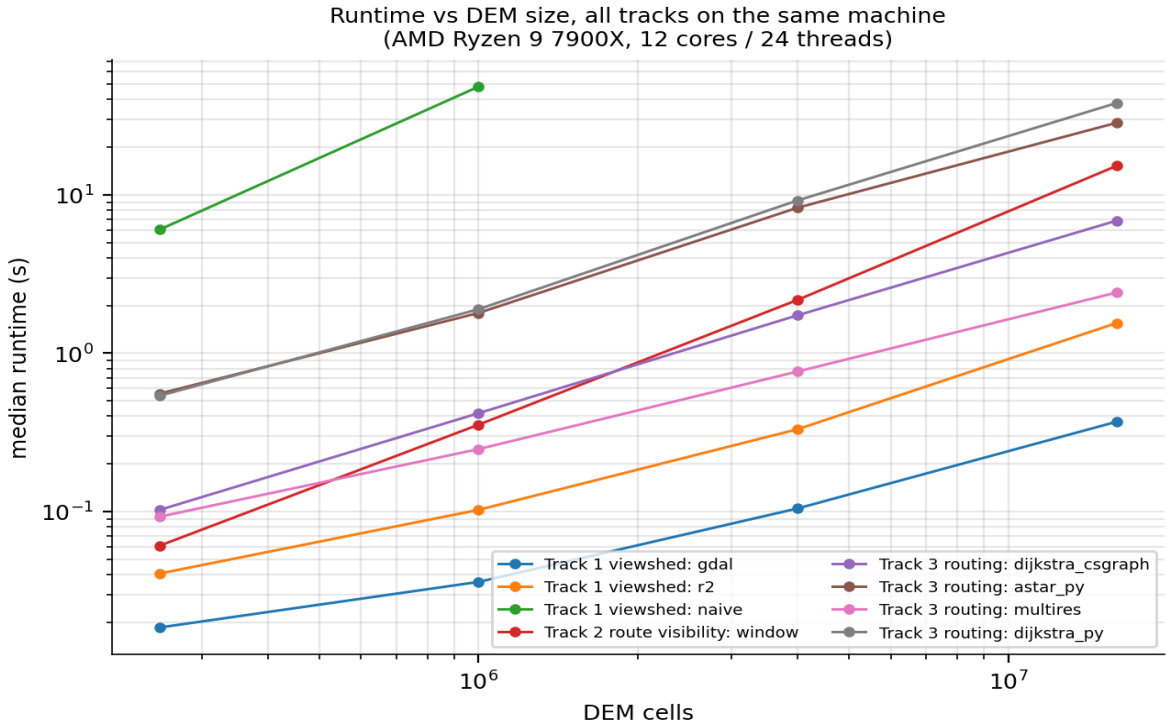
Hidden from the radars is not blind to the ground



Route	km	covered km	P(detect)	worst	aircraft sees km²
Track 2 valley	191	49	100%	R1	1,205
Track 3 shortest	131	50	100%	R1	2,169
Track 3 α = 10	170	4	73%	R2	2,456
Track 3 α = 100	216	2	27%	R1	2,320
Track 3 P ≤ 50%	188	3	50%	R1	2,303
Least covered	215	1	35%	R1	2,945

Track 1's radars and Track 3's detection model score each route; Track 2's engine measures the ground the aircraft itself sees (150 m AGL, 20 km). Hiding from six radar sites does not blind the aircraft: the least-covered route is detected with 35% probability yet sees 2,945 km², 2.4× the valley route.

Three workloads, three cost drivers

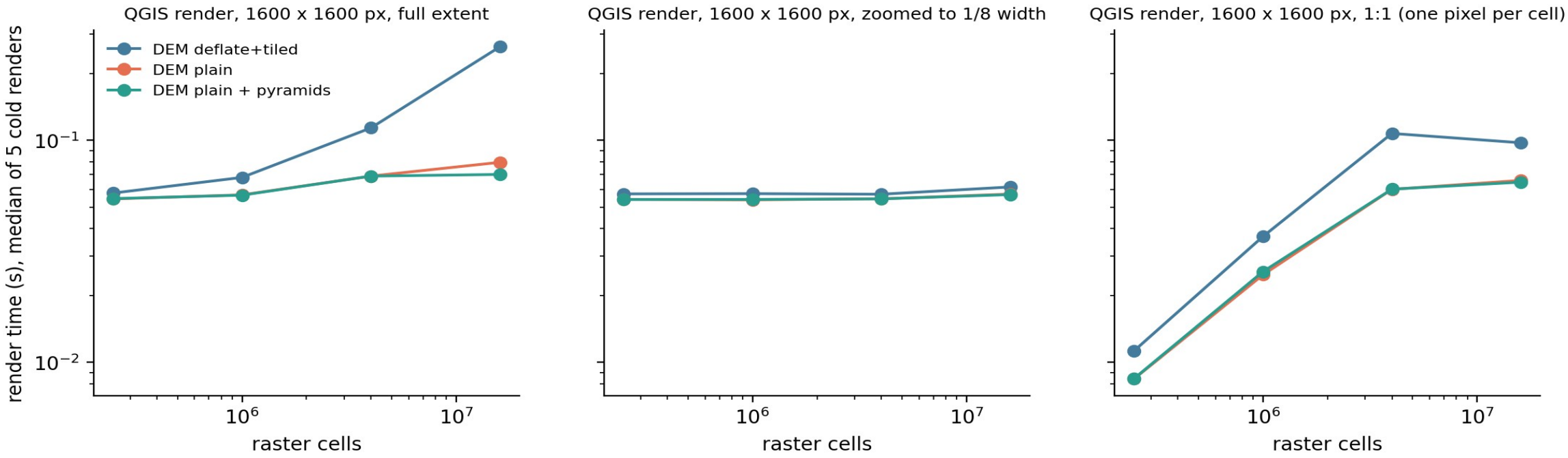


Track	Unit of work	Grows with	Exponent	Best lever
1 Radar	one viewshed	cells in range	0.72	range window, threads
2 Flight	viewshed per position	positions × radius²	1.33	threads, spacing per altitude
3 Corridor	graph search	nodes (N log N)	1.01	C on implicit grid, precompute

Raw runtimes across tracks are not a contest: they do different work. Comparisons that matter are within a track; across tracks we compare what drives the cost.

AMD Ryzen 9 7900X, 12 cores / 24 threads, 31.1 GB · Python 3.12.13, GDAL 3.13.1, NumPy 2.4.6

QGIS cost follows the screen, not the raster



Plain or pyramided rasters draw in tens of milliseconds at any size. Compressed tiled GeoTIFFs without pyramids grow with the raster at full extent (every tile is decompressed). Build pyramids for compressed outputs; the plugin keeps its threat layer in memory.

What we learned

Track 1

Exact viewsheds cost N^1.5; R2 and GDAL stay linear at 99.8% agreement. Read only the range window, run many radars on threads (4.2×), coarsen only with a kappa check.

Track 2

Cost = positions × min(radius², map), fitted within 5%. Sharing the DEM across threads is fastest (exact); spacing should follow altitude. A real camera sees 5–18% of line of sight in the valley; ~8% of a low-flight coverage map is uncertain.

Track 3

Precompute per goal and every start is a lookup; parallel C makes rebuilds sub-second; a certified risk budget; 4× less memory with direction codes; shipped as a QGIS plugin with Processing tools.

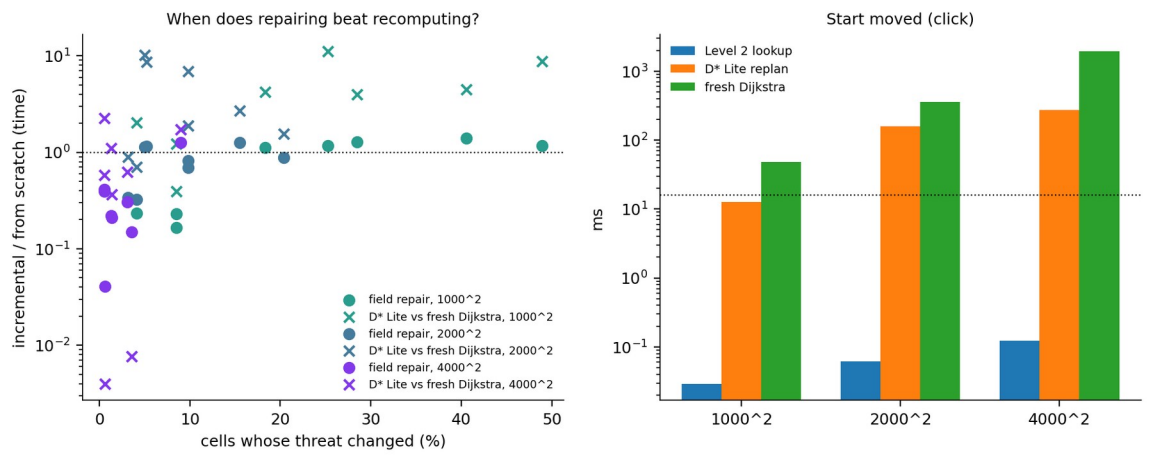
Across tracks: one pipeline on one machine, detection consistent with coverage, and every route scored by all three tracks.

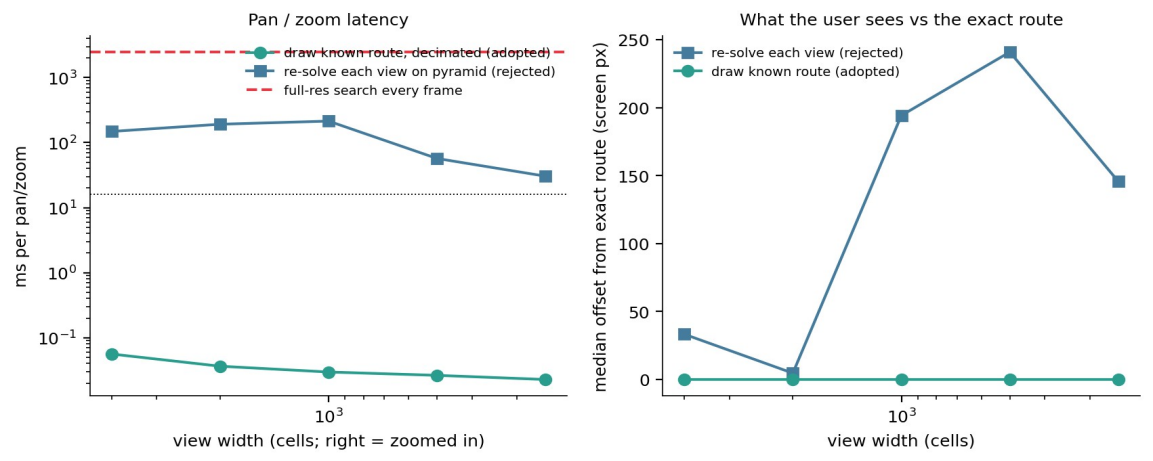
THANK YOU

Questions?

Code, results and the plugin ZIP in the team repository · run.bat -m integration.pipeline reproduces every number

Recompute only what an edit invalidates





Start moved

Field lookup ~0.1 ms vs 2.7 s Dijkstra; D* Lite often needs no work, but the lookup is simpler.

Radar dragged

Recompute that radar's viewshed window (~0.2 s), then rebuild the field in parallel (~0.4 s). Exact.

Pan / zoom

Never search: draw the known route. Re-solving each view: 30–240 ms and often the wrong valley.

From the matplotlib viewer to the plugin with live preview

